A project report on

SMARTPACK: CLOSURE-VERIFIED RFID SMART BAG WITH TOGGLE-BASED ITEM TRACKING

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering

by

ROHITH.T (23MIA1104)

R. SANDEEP KUMAR (23MIA1040)

SASTIKA.M (23MIA1047)



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

October, 2026

ABSTRACT

Forgetting essential items such as a wallet, keys, a charger or documents when leaving home is a common everyday problem. Existing item trackers only help to find an object after the owner has noticed that it is missing. This project presents SmartPack, a low-cost smart bag built around an ESP32 microcontroller that checks the contents of each compartment at the moment the compartment is closed. The prototype has two compartments. Each has its own MFRC522 13.56 MHz RFID reader and a magnetic reed switch, and the design can be extended to more compartments by adding one reader, one reed switch and one wiring-table row for each. Every item carries a passive RFID tag.

Each scan of a tagged item at a compartment's reader represents one crossing of that compartment's opening. The first scan marks the item as inside and the next scan marks it as taken out, so the firmware keeps a simple inside/outside state for every item. When the reed switch shows that the compartment has been closed, the firmware checks that every item assigned to that compartment is marked inside. If any item is not, a red LED and a buzzer alert the user at once, and the identities of the missing items are sent over Bluetooth Low Energy to a Flutter mobile application.

The application adds items by tapping their tags on the reader, shows the live inside/outside status of every item, lets the user rename compartments and delete items, and keeps all item names on the phone, so the bag itself stores only anonymous tag identities. The firmware, the communication protocol and the application have been implemented and tested on a working model, and the core logic and the application's protocol parser are also covered by automated tests.

Keywords: RFID, ESP32, MFRC522, reed switch, Bluetooth Low Energy, Flutter, smart bag, item verification.

TABLE OF CONTENTS

1. PROJECT TITLE..... 1
1.1 Title of the Project..... 1
1.2 Meaning of the Title..... 1
2. PROBLEM STATEMENT..... 1
2.1 The Everyday Problem..... 1
2.2 Why Existing Solutions Are Not Enough..... 1
2.3 Problem Statement..... 2
2.4 Objectives..... 2
3. RESEARCH GAP..... 2
3.1 What Existing Work Covers..... 2
3.2 Gaps Found..... 2
3.3 How SmartPack Answers the Gaps..... 3
4. NOVELTY / PROPOSED CONTRIBUTION..... 3
4.1 Main Ideas..... 3
4.2 From the First Design to the Built Prototype..... 4
4.3 Contributions of the Project..... 4
4.4 Limits of the Novelty Claim..... 4
5. FIELD WORK..... 5
5.1 Purpose of the Field Work..... 5
5.2 Activities Carried Out..... 5
5.3 Requirements Drawn from the Field Work..... 5
6. LITERATURE SURVEY..... 6
6.1 Scope and Method of the Survey..... 6
6.2 Review of Research Papers..... 6
6.3 Comparison of the Surveyed Works..... 11
6.4 Gap Analysis and Position of SmartPack..... 12
7. DESCRIPTION OF PLANNED MODULES..... 13
7.1 Tag Registration Module..... 13
7.2 Closure Sensing Module..... 13
7.3 RFID Scanning and Toggle Module..... 13
7.4 Verification Module..... 14
7.5 Local Feedback Module..... 14
7.6 Wireless Link Module..... 14
7.7 Mobile Application Module..... 14
7.8 Power Management Module..... 14
8. IDENTIFIED METHODOLOGIES..... 15
8.1 Event-Triggered Sensing..... 15
8.2 Crossing-Based Toggle Tracking..... 15
8.3 Set Comparison..... 15
8.4 Hardware and Software Split for Privacy..... 15
8.5 Modular and Table-Driven Design..... 15

8.6 Iterative Development....15
8.7 Testing Methodology....15
8.8 Experimental Validation....16
8.9 Prior-Art Verification....16
9. UML DIAGRAMS....17
9.1 Use Case Diagram....17
9.2 Class Diagram....18
9.3 Sequence Diagram....18
9.4 Activity Diagram....19
9.5 Component Diagram....20
9.6 Deployment Diagram....21
9.7 State Machine Diagram....22
9.8 Communication Diagram....23
9.9 Differences Between the Design-Stage Diagrams and the Built Prototype....24
10. ARCHITECTURE DESIGN....25
10.1 Three-Layer Architecture (Design Stage)....25
10.2 Architecture of the Built Prototype....25
10.3 Main Components and Interfaces....26
10.4 Data Architecture....27
11. ALGORITHMS & FLOWCHARTS....28
11.1 Design-Stage Algorithm and Flowchart....28
11.2 Algorithm 1: Toggle-Based Item Tracking....29
11.3 Algorithm 2: Closure Verification....30
11.4 Algorithm 3: Tap-to-Enrol....31
11.5 Main Loop of the Firmware....32
11.6 Timing Settings and Cost....33
12. HARDWARE DEVELOPMENT....34
12.1 Hardware Overview and Components....34
12.2 Wiring Diagram of the Two-Compartment Prototype....34
12.3 Pin Assignment....35
12.4 Power Supply....36
12.5 Safety Rules....36
12.6 Building the Working Model....36
12.7 Adding More Compartments....39
13. SOFTWARE DEVELOPMENT....40
13.1 Development Tools....40
13.2 Firmware....40
13.3 BLE Communication Protocol....41
13.4 Mobile Application....42
13.5 Software Testing Done So Far....44
14. CLOUD / DATA ANALYTICS / COMPUTER SCIENCE INTEGRATION....45
14.1 Cloud Integration....45
14.2 Computer Science Concepts Used....45
14.3 Data Analytics Possibilities....45
14.4 Privacy and Security....46

15. FUNCTIONAL DEMONSTRATION.....47
15.1 Demonstration Setup.....47
15.2 Demonstration Script.....47
15.3 Observations.....47
16. VALIDATION & DEPLOYMENT.....49
16.1 Validation Completed So Far.....49
16.2 Planned Further Measurements.....49
16.3 Known Limitations and Risks.....50
16.4 Deployment.....50
16.5 Conclusion.....50
16.6 Future Work.....51
REFERENCES.....52

LIST OF FIGURES

Figure 1: UML use case diagram of the system.....17
Figure 2: UML class diagram.....18
Figure 3: UML sequence diagram of closure verification.....19
Figure 4: UML activity diagram of the verification workflow.....20
Figure 5: UML component diagram.....21
Figure 6: UML deployment diagram.....22
Figure 7: UML state machine diagram.....23
Figure 8: UML communication diagram.....24
Figure 9: Complete system architecture (design stage).....25
Figure 10: Architecture of the built SmartPack prototype.....26
Figure 11: Flowchart of the design-stage verification algorithm.....29
Figure 12: Inside/outside state of a tracked item.....30
Figure 13: Flowchart of the closure verification of the built prototype.....31
Figure 14: Sequence of adding an item by tap-to-enrol.....32
Figure 15: Wiring diagram of the two-compartment SmartPack prototype.....35
Figure 16: The assembled SmartPack working model (front view).....37
Figure 17: Electronics of the working model on breadboards.....38
Figure 18: RFID reader (RFID-RC522) fixed inside a compartment.....38
Figure 19: Reed switch and magnet fitted on the zip of a compartment.....39
Figure 20: Screen of the SmartPack mobile application.....43

LIST OF TABLES

Table 1: Gaps and the SmartPack answer....3
Table 2: Contributions....4
Table 3: Field work activities and their outputs....5
Table 4: Comparison of the surveyed works....11
Table 5: Overview of the modules....13
Table 6: Design-stage diagrams compared with the built prototype....24
Table 7: Main architecture components....26
Table 8: Where each piece of data is stored....27
Table 9: Timing and size settings of the firmware....33
Table 10: Hardware components....34
Table 11: ESP32 pin assignment....35
Table 12: Development environment....40
Table 13: LED and buzzer patterns....41
Table 14: Commands from the app to the bag....41
Table 15: Events from the bag to the app....41
Table 16: Software tests....44
Table 17: Computer science concepts used in the project....45
Table 18: Demonstration steps and expected responses....47
Table 19: Validation status....49

LIST OF ACRONYMS

API	Application Programming Interface
APK	Android Package
BLE	Bluetooth Low Energy
GATT	Generic Attribute Profile
GPIO	General-Purpose Input/Output
GPS	Global Positioning System
GSM	Global System for Mobile Communications
HF	High Frequency (13.56 MHz)
IEEE	Institute of Electrical and Electronics Engineers
IoT	Internet of Things
LED	Light-Emitting Diode
Li-Po	Lithium Polymer
MTU	Maximum Transmission Unit
NFC	Near-Field Communication
RFID	Radio-Frequency Identification
RST	Reset (pin)
SPI	Serial Peripheral Interface
SS / CS	Slave Select / Chip Select
UHF	Ultra High Frequency
UID	Unique Identifier (of an RFID tag)
UML	Unified Modeling Language
USB	Universal Serial Bus
UUID	Universally Unique Identifier

1. PROJECT TITLE

1.1 Title of the Project

SmartPack: Closure-Verified RFID Smart Bag with Toggle-Based Item Tracking

SmartPack is a smart bag that checks its own contents. When the user closes a compartment, the bag checks that every item that belongs in that compartment is really inside. If something is missing, the bag warns the user at once with a red light and a buzzer, and it tells the mobile phone exactly which item is missing.

1.2 Meaning of the Title

Each part of the title describes one idea of the project:

- **Closure-verified** means the check happens at the moment a compartment is closed, not all the time.
- **RFID** means every item carries a small passive tag with its own identity number, and a reader inside the bag reads it.
- **Toggle-based item tracking** means each scan of an item flips its state. The first scan means the item has gone **in**, the next scan means it has come **out**, and so on. Each scan represents one crossing of the compartment opening.
- **Smart bag** means the bag has its own small computer, lights, a buzzer and a Bluetooth link to the phone.

2. PROBLEM STATEMENT

2.1 The Everyday Problem

Most people carry a bag with a few things they cannot do without: a wallet, house keys, a phone charger, an identity card, a laptop or a notebook. Forgetting even one of them can mean a wasted trip back home, a missed class or a stressful day. The usual way to avoid this is a mental checklist, but a mental checklist fails exactly when a person is in a hurry or thinking about something else.

2.2 Why Existing Solutions Are Not Enough

Bluetooth item trackers such as Tile and AirTag help to find an item **after** the owner has noticed it is missing. They do not check the bag before the owner leaves. Reminder systems based on RFID do exist, but many of them depend on readers fixed at a door or in a room [1], [2], so they only work at that one place. Other systems for RFID baggage tracking

are built for airports and warehouses [3], [4], which is far more than one person needs for one bag.

2.3 Problem Statement

There is no simple, low-cost and personal system that checks the contents of a bag automatically at the moment the bag is closed, tells the user right away which item is missing, and works for a bag with more than one compartment.

2.4 Objectives

- Build an ESP32-based bag in which every compartment has its own RFID reader and its own reed switch.
- Treat each RFID scan as one crossing of the compartment opening, so that the first scan of an item means **in** and the next scan means **out**.
- Check, when a compartment is closed, that every item assigned to it is inside, and warn the user at once with LEDs and a buzzer.
- Send the identity of any missing item to a mobile application over Bluetooth, without the bag ever storing item names.
- Build a mobile application that adds items by tapping their tags, shows live in/out status, deletes items and shows missing-item alerts.
- Test the system with a clear set of measurements and describe how it can be deployed.

3. RESEARCH GAP

3.1 What Existing Work Covers

The related work falls into a few groups. Reminder systems use fixed readers at doors or in rooms to warn people about forgotten objects [1], [2], [5]. Smart bag projects add RFID, GPS or other sensors to a bag [6], [7], [8], [9], [10], [11]. One system watches items in a vehicle or on equipment using a UHF base unit [12]. Large-scale RFID work looks at airports and at missing-tag detection across many readers [3], [4]. Section 6 reviews each of these papers in detail.

3.2 Gaps Found

From the papers reviewed, we found five gaps that SmartPack tries to close:

- **Gap 1: No closure trigger.** None of the reviewed works uses the closing of a compartment as the event that starts the check. Most scan all the time or when an item is placed.

- **Gap 2: No per-compartment checking.** The reviewed bags are treated as one unit. None is described as having a separate reader and reed switch for each compartment.
- **Gap 3: No simple in/out state.** Few systems keep a clear inside/outside state for each item based on its crossings.
- **Gap 4: Limited phone support.** Only some works have a phone application, and they mainly show alerts or logs rather than manage the items of a bag.
- **Gap 5: Names stored on the device.** Most systems do not separate raw tag identities (on the bag) from item names (on the phone), which matters if a bag is lost.

These gaps are based on the abstracts and descriptions of the papers that we could read. They should be checked again against the full texts.

3.3 How SmartPack Answers the Gaps

Table 1: Gaps and the SmartPack answer

Gap	Typical existing approach	SmartPack approach
1	Continuous scanning or scan when an item is placed	The reed switch reports that a compartment is closed, and that event starts the check
2	One reader for the whole bag	One reader and one reed switch for each compartment
3	List of tags currently seen	One inside/outside state per item, flipped by each scan
4	Alerts or logs on the phone	App adds, lists, deletes and corrects items, and names the missing ones
5	Item names stored with the device	Bag stores only tag IDs; names stay on the phone

4. NOVELTY / PROPOSED CONTRIBUTION

4.1 Main Ideas

- **Closure-triggered verification.** The check is done when the user closes a compartment, which is the moment the user is about to leave.
- **Crossing-based toggle tracking.** Each scan is read as one crossing of the opening. This keeps the reader small and short-range, and still gives a clear inside/outside state for every item.
- **Per-compartment sensing.** Each compartment has its own reader and reed switch, so a bag with several compartments is checked compartment by compartment.

- **Privacy by design.** The bag knows only tag IDs. The names, such as “Wallet” or “Charger”, live on the phone.
- **Tap-to-enrol.** A new item is added by naming it in the app and tapping its tag on the reader.
- **Scalable design.** The number of compartments is a single setting in the firmware and the app, so the design is not limited to two.

4.2 From the First Design to the Built Prototype

The first design, called ZipCheck, used one reader that scanned all tags for two to three seconds after a zip was closed, with the ESP32 sleeping in between. While building the prototype we refined the design. The final version has one reader and one reed switch for each compartment, treats every scan as a crossing, and keeps the ESP32 awake so that scans and the phone link can be handled at any time. The final wiring also uses a buzzer for audible alerts. Section 9 explains the differences between the original UML diagrams and the built prototype.

4.3 Contributions of the Project

Table 2: Contributions

Area	Contribution
Hardware	Two-compartment circuit with shared SPI bus, separate reader pins, reed switches, LEDs, buzzer and a battery power chain
Firmware	ESP32 program that debounces reed switches, polls readers, toggles item states, verifies compartments and stores data in flash
Protocol	A short text protocol over BLE for commands and events
Mobile app	Flutter app with pairing, enrol, live status, delete, rename and missing-item alerts
Testing	PC unit tests for the core logic, parser tests for the app, and a bench trial plan
Documents	UML diagrams, architecture, algorithms, wiring diagram and this report

4.4 Limits of the Novelty Claim

The novelty claim in this report is limited to the works reviewed in Section 6. A wider search of research databases and patents should still be completed before the claim is made without any limit.

5. FIELD WORK

5.1 Purpose of the Field Work

The field work was done to understand the problem in real life before designing the system, and to turn what we learned into clear requirements. It had three parts: studying how people forget things and what tools they use today, studying the components and prior work, and building and testing the prototype.

5.2 Activities Carried Out

Table 3: Field work activities and their outputs

Activity	What was done	Output
Problem study	Looked at how people forget items when leaving and how they try to avoid it today	Problem statement (Section 2)
Literature and product study	Read research on RFID bags, reminder systems and Bluetooth, and looked at commercial trackers	Literature survey (Section 6)
Component study	Compared options for the controller, the reader and the closure sensor, and chose ESP32, MFRC522 and reed switches	Component list (Section 12)
Circuit design and assembly	Planned the pin map and built the two-compartment working model	Wiring diagram and working model (Section 12)
Software development and testing	Wrote the firmware and the mobile app, ran unit tests and builds	Sections 13 and 16

5.3 Requirements Drawn from the Field Work

The problem study and the literature led to the following requirements for SmartPack:

- The bag must give an alert on its own, even when the phone is not nearby.
- The check must happen at the right moment, which is when a compartment is closed.
- The system must be low cost and use common, easy-to-find parts.
- Items must be easy to add and remove without changing any hardware.
- The bag must not reveal what a person carries if it is lost.
- The design must work for more than one compartment.

6. LITERATURE SURVEY

6.1 Scope and Method of the Survey

We looked for papers on five topics: RFID-based smart bags, reminder systems for forgotten items, RFID baggage tracking, missing-tag detection, and Bluetooth Low Energy as the link to a phone. Thirteen papers were selected because they are closely related to one or more parts of SmartPack. Each paper is discussed under six headings: authors and year, research problem, methodology, key findings, limitations or gap, and relevance to this project. The details come from the published abstracts and descriptions of the papers, so the findings are described in general terms.

6.2 Review of Research Papers

6.2.1 Reminding About Tagged Objects Using Passive RFIDs

Authors and year: G. Borriello, W. Brunette, M. Hall, C. Hartung and C. Tangney, 2004. [1]

Research problem: People often leave behind objects they own, and a system is needed to remind them.

Methodology: Passive RFID tags are attached to objects that people often misplace. Fixed readers placed in areas that people visit detect the tags, and a personal server decides whether a reminder is needed. Reminders are shown as text and beeps on a small wristwatch-sized device.

Key findings: The authors present the design as scalable and as better for user privacy than earlier approaches.

Limitations / gap: It needs fixed readers in the places the user visits, so it only works where such readers are installed. It does not check the contents of a bag.

Relevance to SmartPack: It shows that passive RFID tags are a practical way to remind people about forgotten items. SmartPack moves the reader into the bag itself.

6.2.2 An RFID-Based Reminder System for Smart Home

Authors and year: H.-H. Hsu, C.-N. Lee and Y.-F. Chen, 2011. [2]

Research problem: People forget items when they leave the house.

Methodology: An RFID reader detects the objects that a user takes out of the house. The system builds a reminder list from the user's own history and from events in the user's calendar, and the user can mark objects that are not needed.

Key findings: The system gives the user a list of objects to remember and improves it using the user's feedback.

Limitations / gap: The reader is fixed at one place, and the quality of the reminders depends on stored history and calendar data.

Relevance to SmartPack: It shows the value of user feedback and of a list for each day. SmartPack keeps a list for each compartment and checks it when the compartment is closed.

6.2.3 An Intelligent RFID System (IPURSE)

Authors and year: M. K. Watfa, M. Kaur and R. F. Daruwala, 2011. [11]

Research problem: Items carried in a purse or handbag are easily forgotten or lost.

Methodology: RFID and NFC are combined on a mobile platform to track the items in a handbag. The system alerts the user when something is missing and uses context such as the weather and the calendar.

Key findings: The authors describe the design, a working prototype and experiments, and they discuss the advantages and disadvantages of the system.

Limitations / gap: The check is not tied to the closing of the bag, and it does not treat compartments separately.

Relevance to SmartPack: It is one of the closest systems to SmartPack in purpose. SmartPack differs by using a closure trigger and one reader for each compartment.

6.2.4 RFID-Based Interactive Log Bag for Poor Memory Prevention

Authors and year: M. S. Mohd Yazed and F. Mahmud, 2016. [10]

Research problem: People with poor memory forget the essential items they need when travelling or going to work.

Methodology: A bag built with an Arduino Mega 2560 and an RFID system logs the tagged objects when they go in and out. The log is saved on an SD card and can be shown in an Android application, and an RGB LED strip reminds the user of forgotten items.

Key findings: The work produces a working bag that records in and out events and reminds the user about forgotten items.

Limitations / gap: The description does not mention separate readers for each compartment or a check triggered by closing the bag.

Relevance to SmartPack: It is the closest work to the in/out idea of SmartPack. SmartPack adds per-compartment readers, a closure check, and a Bluetooth app that adds and manages items.

6.2.5 Smart Bag Based on RFID and Internet of Things

Authors and year: A. S. Chauhan and D. Nigam, 2021. [6]

Research problem: School children forget books and other items, and parents want to know where the bag is.

Methodology: An RFID reader reads the tags on books and items. A NodeMCU compares them with the schedule for the day, and an HX711 load cell is used as a second sensor. A GPS module shares the location and a geofence raises an alert.

Key findings: The bag alerts the user when items from the day's schedule are missing and can share its location.

Limitations / gap: It depends on a saved schedule, uses extra sensors that add cost and power use, and is not triggered by closing the bag.

Relevance to SmartPack: It shows the benefit of combining RFID with another sensor. SmartPack uses a simple reed switch as its second sensor.

6.2.6 RFID Based Smart Personal Baggage System

Authors and year: P. Kasun and S. M. Vidanagamachchi, 2018. [7]

Research problem: Personal belongings in backpacks, handbags and travel bags need to be tracked and protected from theft.

Methodology: An RFID system is built into the bag for real-time tracking. A dynamic transmit-power algorithm keeps the reading range limited to the bag, and a custom patch antenna is used.

Key findings: The authors describe a design that limits the reading range to the bag and supports tracking and anti-theft use.

Limitations / gap: The focus is on radio range and the antenna. The abstract does not describe a per-compartment check or a phone-managed item list.

Relevance to SmartPack: It points out the problem of reading tags that are outside the bag. SmartPack uses short-range readers at each compartment opening.

6.2.7 Real-Time Detection and Notification of Forgotten Items Using UHF RFID

Authors and year: A.-M. V. M. Calo, T. H. O. Bato, C. J. Abragan and E. S. Tutor, 2018. [12]

Research problem: People forget items when they leave a vehicle or equipment.

Methodology: A UHF RFID base unit with a controller, a pocket Wi-Fi router and a power source is used. Tags are fixed to the items, and a notification is sent when an item is no longer detected within 1.5 metres of the base unit.

Key findings: The work shows a base-unit and notification system that can warn the user about forgotten items.

Limitations / gap: UHF hardware is larger and needs more power, range-based detection can be affected by the surroundings, and the system is made for vehicles and equipment, not for bags.

Relevance to SmartPack: It supports the idea of a base device that notifies a phone. SmartPack uses HF readers with a very short range at each compartment.

6.2.8 Localisation of Forgotten Items Using RFID Technology

Authors and year: J. Hallberg, C. Nugent, R. Davies and M. Donnelly, 2009. [5]

Research problem: Misplacing items becomes more common with age and is especially hard for people with memory problems.

Methodology: RFID-tagged items inside a home are located by comparing their signal strength with the signal of fixed reference tags.

Key findings: The authors report acceptable location estimates when the reference tags are few, and they say that more work is needed to measure the accuracy.

Limitations / gap: It tries to find where an item is, not whether it is present. It needs reference tags in the home.

Relevance to SmartPack: It is useful background on RFID for forgotten items. SmartPack answers the simpler question of whether an item is present or absent.

6.2.9 Multifunctional Bag Monitoring System

Authors and year: R. Varma, P. Pavshe, A. Bhadane and S. Pagare, 2018. [8]

Research problem: A bag and its contents need protection from theft.

Methodology: The system uses a Raspberry Pi with GSM, GPS, a camera, an electromagnetic sensor and a battery. It sends an SMS when the bag leaves a geo-fenced area, logs each opening and closing of the bag, and an Android application is mentioned for tracing the bag.

Key findings: The authors propose a multi-function bag monitor that combines location, an alert and a log of openings.

Limitations / gap: The aim is security and not item checking, it does not identify the items inside, and a Raspberry Pi needs more power than a microcontroller.

Relevance to SmartPack: It shows that the opening and closing of a bag can be sensed and used as an event. SmartPack uses closure to start an item check.

6.2.10 An IoT-Based Smart Bag with Integrated Security System and Rainwear

Authors and year: S. M. Antad, O. Khanvilkar, C. Kharare, J. Kharat and K. Kharat, 2023. [9]

Research problem: Bags can be stolen, and their contents can be damaged by rain.

Methodology: A GPS module and an Arduino board are used to track the bag. A power bank is included for charging, and a raincoat is built into the back section of the bag.

Key findings: The authors present a prototype that combines tracking, security features and rain protection.

Limitations / gap: It does not check which items are inside the bag. Location tracking with GPS also uses more power.

Relevance to SmartPack: It shows the interest in multi-function smart bags built on low-cost boards. SmartPack addresses the missing item-check function.

6.2.11 A High Time-Efficient Missing Tag Detection Scheme for Integrated RFID Systems

Authors and year: K. Guo, X. Xie, H. Qi and K. Li, 2022. [4]

Research problem: Finding which tags are missing is slow in integrated systems that have many readers and different kinds of tags.

Methodology: The authors propose a protocol called MMTD. It runs detection for the different tag categories in parallel, divides the work using the spatial spread of the tags, and adjusts the reader's time frame to the number of tags.

Key findings: The authors report that the scheme improves the time efficiency of missing-tag detection.

Limitations / gap: It is made for large systems, such as warehouses, and works at the radio-protocol level.

Relevance to SmartPack: It supports the idea of comparing an expected set with a detected set. For a bag with a few items, SmartPack uses a direct comparison on the microcontroller.

6.2.12 Overview and Evaluation of Bluetooth Low Energy

Authors and year: C. Gomez, J. Oller and J. Paradells, 2012. [13]

Research problem: The capabilities and performance limits of Bluetooth Low Energy (BLE) needed to be understood.

Methodology: The paper describes the main features and applications of BLE and studies how the key parameters affect performance, using theory, simulation and experiments.

Key findings: The authors report that the lifetime of a BLE device on a coin cell can range from about 2 days to 14.1 years depending on the settings, and that the number of slaves per master ranges from 2 to 5,917. Under a high bit error rate the latency can grow by up to three orders of magnitude.

Limitations / gap: It is a general study of the technology and not about bags. Performance depends on the settings that are chosen.

Relevance to SmartPack: It supports the choice of BLE for the phone link of a battery-powered bag.

6.2.13 Baggage Tracing and Handling System Using RFID and IoT for Airports

Authors and year: A. K. Singh, S. Meshram, T. Gujar and P. R. Wankhede, 2016. [3]

Research problem: Baggage handling at airports takes time and baggage can be mishandled.

Methodology: Smart RFID tags are attached to bags and tracked through an IoT system with a cloud server. A prototype was tested at two check-in and check-out locations.

Key findings: The authors aim to reduce the time for baggage handling and to improve security.

Limitations / gap: It is designed for an airport and needs a network and a cloud server. It does not check the contents of a personal bag.

Relevance to SmartPack: It shows that RFID baggage tracking is a mature idea. SmartPack works offline and for one person.

6.3 Comparison of the Surveyed Works

Table 4 compares the works by their setting, technology and alerts, and shows whether any of them checks the items at closure.

Table 4: Comparison of the surveyed works

Ref.	Setting	Main technology	Alert or interface	Check at closure?
[1]	Rooms and places	Passive RFID, fixed readers	Wrist device	No
[2]	Home doorway	RFID reader, history, calendar	Reminder list	No
[11]	Handbag	RFID and NFC, mobile platform	Alert on phone	No
[10]	Bag	RFID, Arduino Mega	Android app, LED strip	No
[6]	School bag	RFID, load cell, NodeMCU, GPS	Alerts, location	No
[7]	Bag	RFID, patch antenna	Tracking, anti-theft	No
[12]	Vehicle, equipment	UHF RFID base unit, Wi-Fi	Notification	No
[5]	Home	RFID, reference tags	Location estimate	No
[8]	Bag	Raspberry Pi, GPS, GSM, sensor	SMS, Android app	Logs opening and closing only
[9]	Bag	GPS, Arduino	Location	No
[4]	Warehouse	Missing-tag protocol	Not applicable	Not applicable
[13]	General	Bluetooth Low Energy	Not applicable	Not applicable
[3]	Airport	RFID, IoT, cloud	Tracking	No
–	**SmartPack**	RFID per compartment, reed switch, ESP32, BLE	LEDs, buzzer, Flutter app	**Yes**

6.4 Gap Analysis and Position of SmartPack

Table 4 shows that most of the reviewed systems keep the reader on, depend on fixed infrastructure, or aim at security and tracking and not at item checking. The closest works, the log bag and IPURSE, show that in/out events and handbag checking are useful ideas. None of the reviewed papers is described as combining a closure trigger, one reader for each compartment, an in/out state for each item and a phone application that manages the items. This is the position that SmartPack takes.

7. DESCRIPTION OF PLANNED MODULES

SmartPack is divided into eight modules. Table 5 gives an overview, and the sections that follow describe each module. All eight modules have been implemented in the prototype.

Table 5: Overview of the modules

Module	Input	Output	Runs on
7.1 Tag registration	Name from the user, tag scan	Tag ID stored with a name	Phone and ESP32
7.2 Closure sensing	Reed switch level	Debounced open/closed event	ESP32
7.3 RFID scanning and toggle	Tag ID from a reader	In/out state change	ESP32
7.4 Verification	Item states at closure	List of missing items	ESP32
7.5 Local feedback	Events from the modules above	LED and buzzer patterns	ESP32
7.6 Wireless link	Events and commands	BLE messages	ESP32 and phone
7.7 Mobile application	BLE events, user taps	Item list, alerts	Phone
7.8 Power management	Li-Po battery	Stable 5 V and 3.3 V	Hardware

7.1 Tag Registration Module

This module links each tag's identity number to a name chosen by the user, such as "Wallet". The user types the name in the app and taps the tag on the reader. The bag stores the tag ID in its list for that compartment, and the phone stores the name. A new item starts as outside, so its first scan puts it inside.

7.2 Closure Sensing Module

Each compartment has a magnetic reed switch and a magnet. The switch is read with an internal pull-up, so a closed compartment reads LOW. A new level must stay steady for 60 milliseconds before it is accepted, which removes the effect of contact bounce. The module only reports the opening and closing of the compartment.

7.3 RFID Scanning and Toggle Module

Each compartment has an MFRC522 reader. The firmware polls the readers in turn, reads the tag ID and toggles the state of the item: outside becomes inside, and inside becomes outside. Scans of the same tag within 1.5 seconds are ignored, and scans at a closed compartment are not counted.

7.4 Verification Module

When a compartment is closed, this module collects every assigned item that is not marked inside. If the list is empty, all items are present. If the compartment has no assigned items, nothing is checked. The module needs no network and no cloud service.

7.5 Local Feedback Module

The module drives the green LED, the red LED and the buzzer. A green light means a scan was accepted or a compartment is verified, and a red light with beeps means an item is missing, a tag is unknown or a reader has a fault. The patterns are played without blocking the rest of the program.

7.6 Wireless Link Module

The ESP32 acts as a BLE server. The phone writes text commands, and the bag sends text events as notifications. Commands from the phone are placed in a queue and are carried out by the main loop, and events are sent a short time apart so that none is lost.

7.7 Mobile Application Module

The Flutter application keeps the item names and compartment names, shows each item’s in/out status, adds items by tap-to-enrol, deletes items, corrects an item’s state and shows a dialog that names the missing items. It synchronises with the bag each time it connects.

7.8 Power Management Module

A Li-Po battery is charged by a TP4056 module. An MT3608 boost converter raises the voltage to about 5 V for the ESP32 VIN pin, and the readers are powered from the ESP32’s 3.3 V pin. All grounds are shared.

8. IDENTIFIED METHODOLOGIES

8.1 Event-Triggered Sensing

Instead of checking the bag all the time, SmartPack lets the closing of a compartment start the check. This matches the moment that matters to the user, which is just before leaving.

8.2 Crossing-Based Toggle Tracking

A short-range reader at the compartment opening cannot tell the direction in which an item moves. So the firmware uses the order of the scans: the first scan means in, the next means out. This gives one state for every item with very little hardware.

8.3 Set Comparison

The decision is a simple set operation. The missing set is the set of assigned items that are not marked inside. It is small, fast and runs fully on the microcontroller.

8.4 Hardware and Software Split for Privacy

The bag works only with tag IDs. The mapping from an ID to a name is kept on the phone. If the bag is lost, nobody can learn from the bag what its owner carried.

8.5 Modular and Table-Driven Design

Everything that belongs to a compartment is addressed by a compartment number, and the wiring of all compartments is listed in one table. This keeps the code short and allows more compartments to be added by a change of settings.

8.6 Iterative Development

The work was done in stages. First the RFID reading and the reed switch were tested on their own. Then the toggle logic and the local feedback were added, then the Bluetooth link and the mobile application, and finally the second compartment and the extension to more compartments. Each stage was tested before the next one was added.

8.7 Testing Methodology

The decision logic is written without any dependence on the hardware, so it can be tested on a normal computer. The application’s parser for bag events is also tested on its own. The same text commands work over the USB serial port, so the bag can be tested with real tags and switches before the application is used.

8.8 Experimental Validation

The finished prototype will be tested with repeated trials, measuring the accuracy of the toggling, the correctness of the closure alert, false alarms, missed detections, the delay from closure to alert, the reading distance and the battery run time. Section 16 describes the plan.

8.9 Prior-Art Verification

Before the novelty claim is final, research databases and patent records should be searched for closely related terms such as closure-triggered RFID checking and RFID checklist bags.

9. UML DIAGRAMS

The system was modelled with eight standard UML diagrams [14]. They describe the system from different points of view: who uses it, which classes it has, how its parts talk to each other over time, how it works step by step, how it is built from components, where it runs, and how its states change. These diagrams were prepared in the design stage of the project for the closure-triggered concept, and they are shown here redrawn with larger text so that they can be read easily. Section 9.9 lists the points where the built prototype differs from them.

9.1 Use Case Diagram

Figure 1 shows the two actors of the system, the User and the Mobile App, and what each can do. The user registers an RFID item, assigns it to a bag, closes the bag to trigger a scan and views the local alert. The mobile app takes part in assigning items, comparing the expected and detected items, and receiving the missing-item notification.

UML 1 – Use Case Diagram

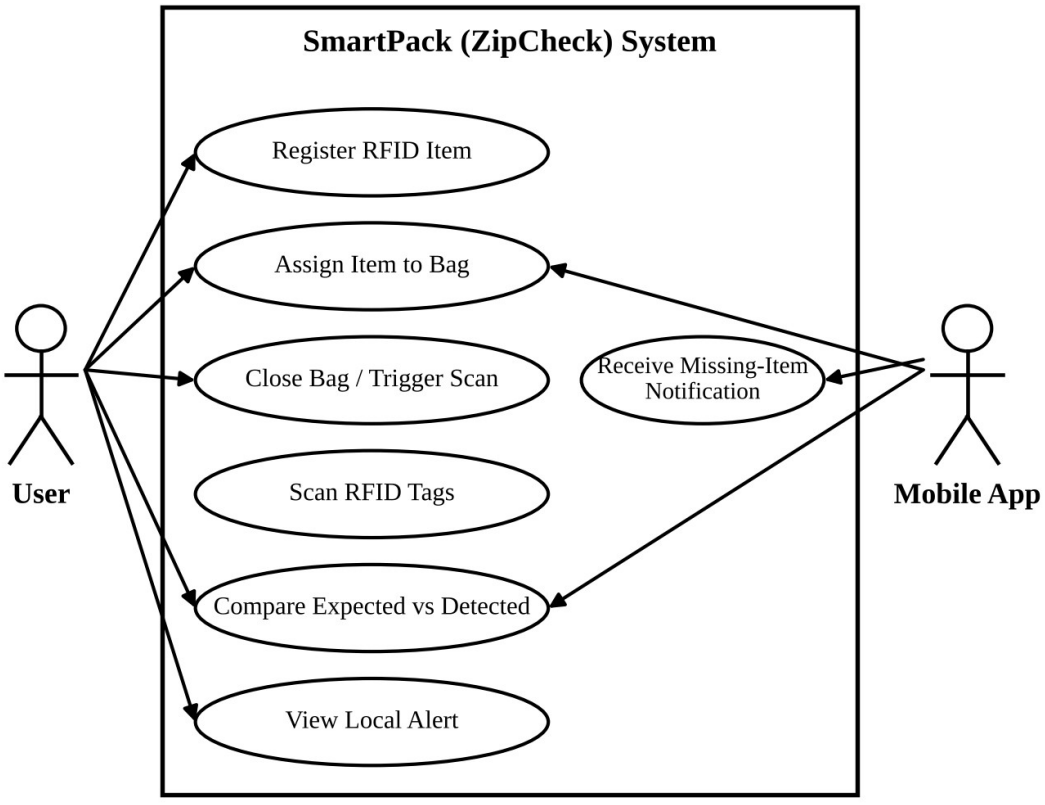


Figure 1: UML use case diagram of the system

9.2 Class Diagram

Figure 2 shows the main classes. A **User** owns **Items** and an Item is assigned to a **Bag**, which keeps the expected set of items. The **VerificationController** compares the expected set with the detected set and finds the missing items. The **RFIDReader** and the **ReedSwitch** supply the scan data and the closure trigger, the **AlertModule** drives the local alerts, and the **MobileApp** turns missing tag IDs into names and notifies the user.

UML 2 – Class Diagram

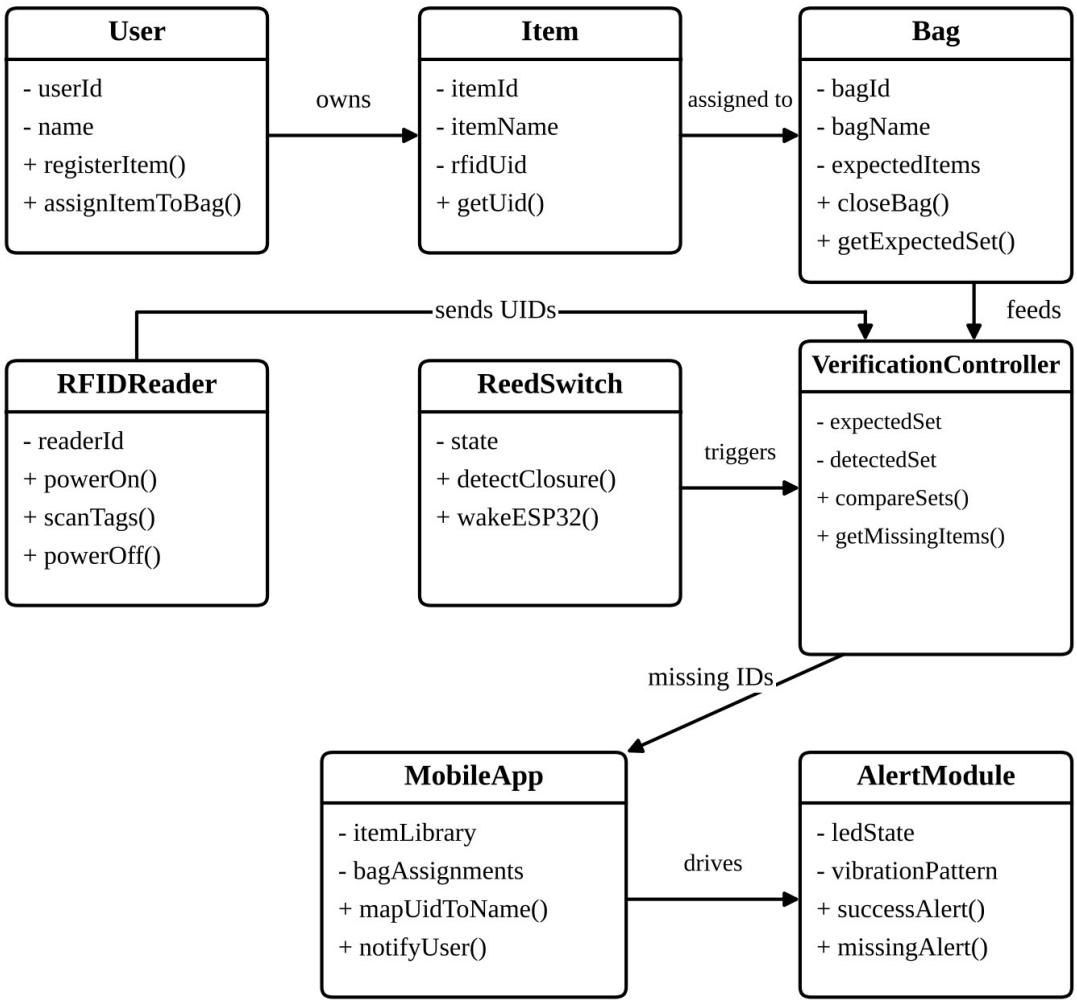


Figure 2: UML class diagram

9.3 Sequence Diagram

Figure 3 shows the order of events after the user closes the bag. The reed switch sends a wake signal to the ESP32. The ESP32 powers the MFRC522, scans for tag IDs and gets the detected set back. It compares the expected set with the detected set. If nothing is missing, it shows the green LED and a short buzz. Otherwise it sends the missing IDs to the app, which maps them to names and notifies the user, while the bag gives the local red alert.

UML 3 – Sequence Diagram: Closure Verification

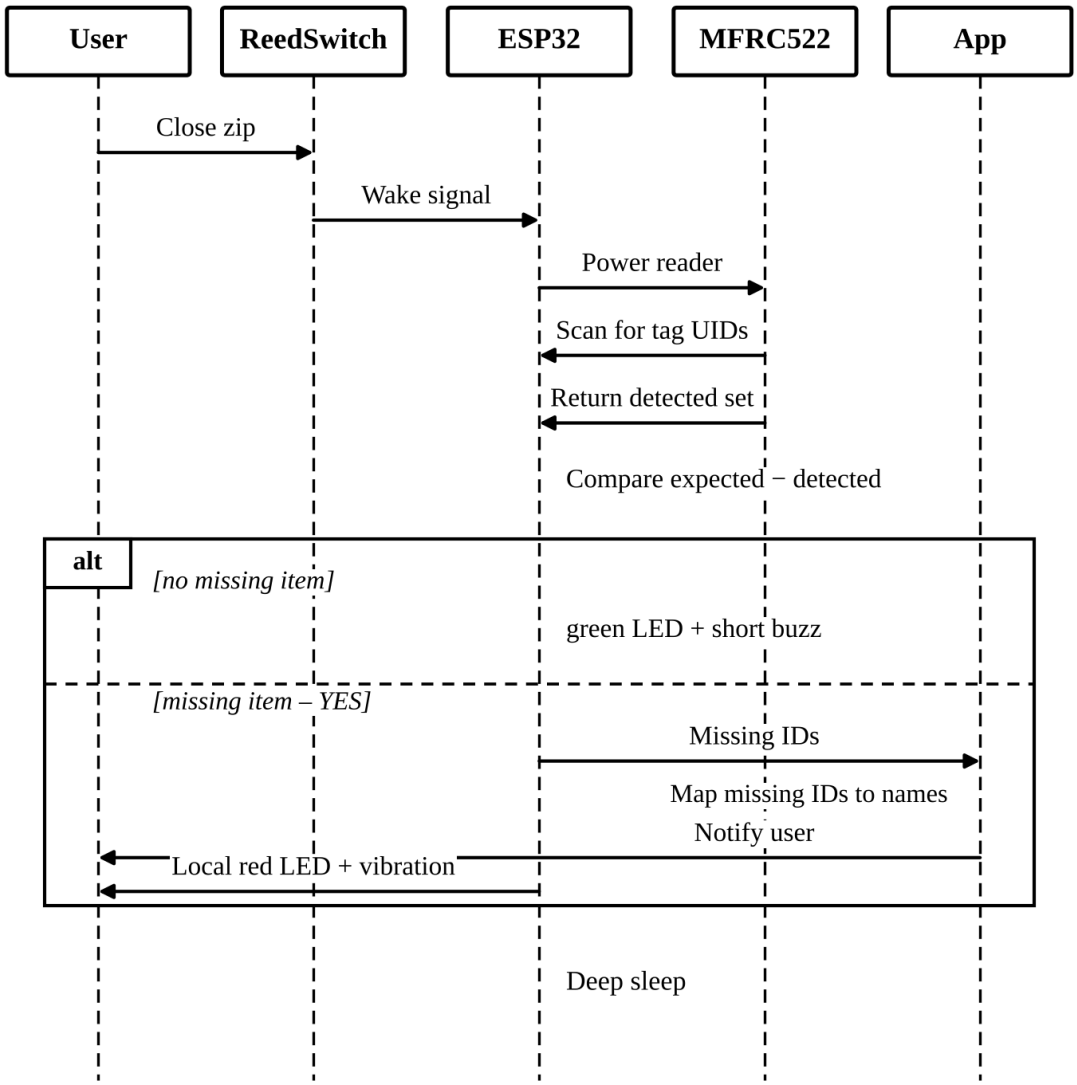


Figure 3: UML sequence diagram of closure verification

9.4 Activity Diagram

Figure 4 follows the verification workflow from the start. The ESP32 waits, the reed switch detects the closure, the ESP32 wakes up and scans, the detected IDs are collected and compared with the expected ones, and the diagram then branches on one decision: is anything missing? If not, the success indication is given. If yes, the red alert is given and the app is informed.

UML 4 – Activity Diagram

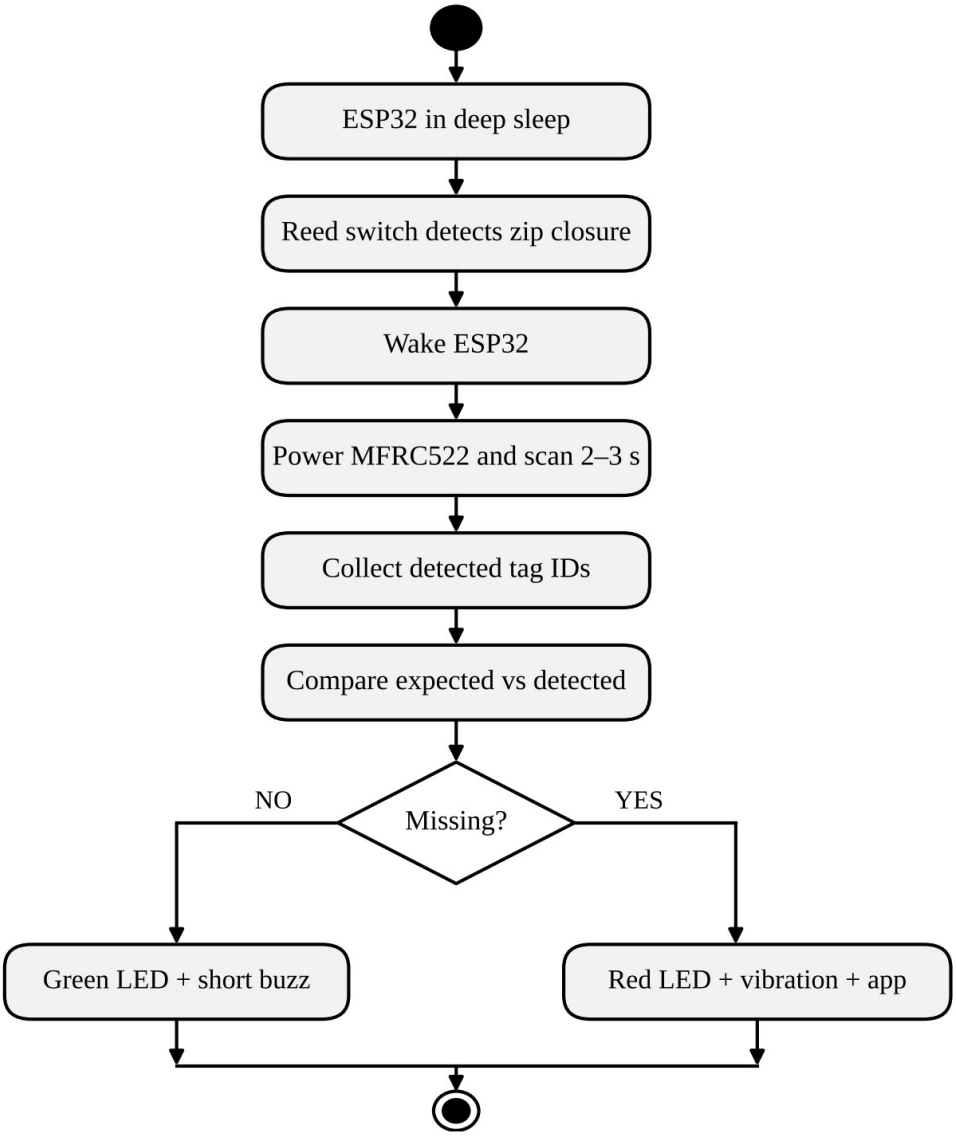


Figure 4: UML activity diagram of the verification workflow

9.5 Component Diagram

Figure 5 shows the hardware and software parts and how they connect: the RFID tags, the reed switch and magnet, the ESP32 controller, the MFRC522 reader, the local alert module, the mobile app, and the battery with its charger. The links are labelled with the interface used, such as SPI between the controller and the reader and BLE between the controller and the app.

UML 5 – Component Diagram

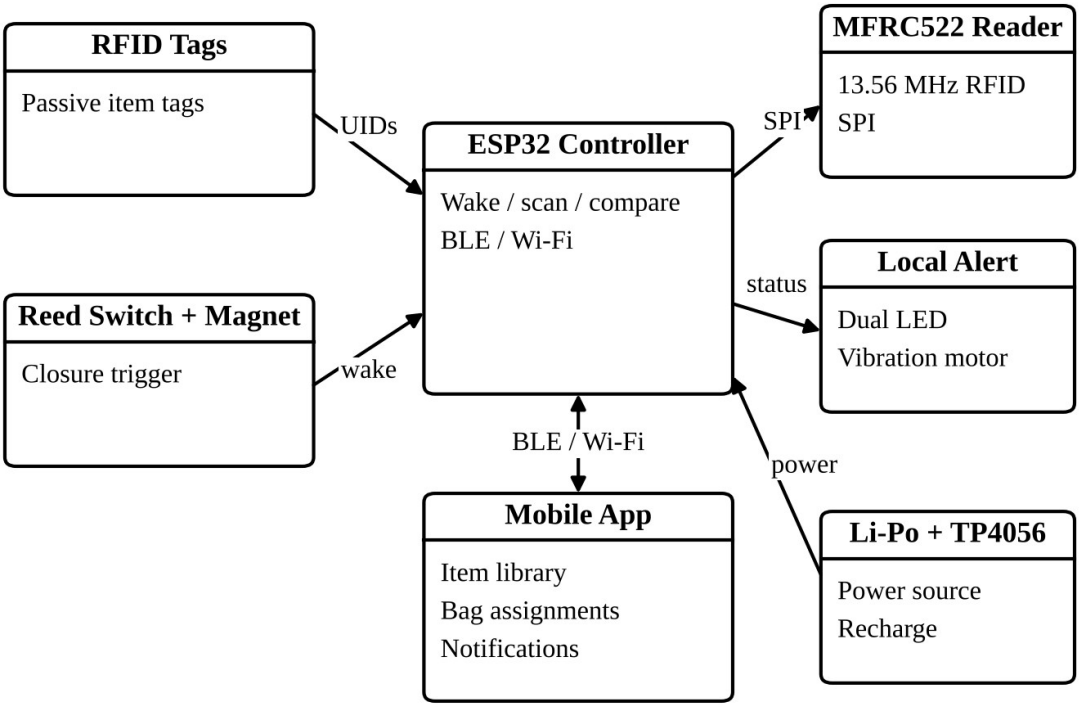


Figure 5: UML component diagram

9.6 Deployment Diagram

Figure 6 shows where each part runs. The smart bag device holds the ESP32, the reader, the reed switch and the power parts. The smartphone holds the application with the item names and the bag assignments. A wireless link joins the two, and only the small mismatch message travels over it.

UML 6 – Deployment Diagram

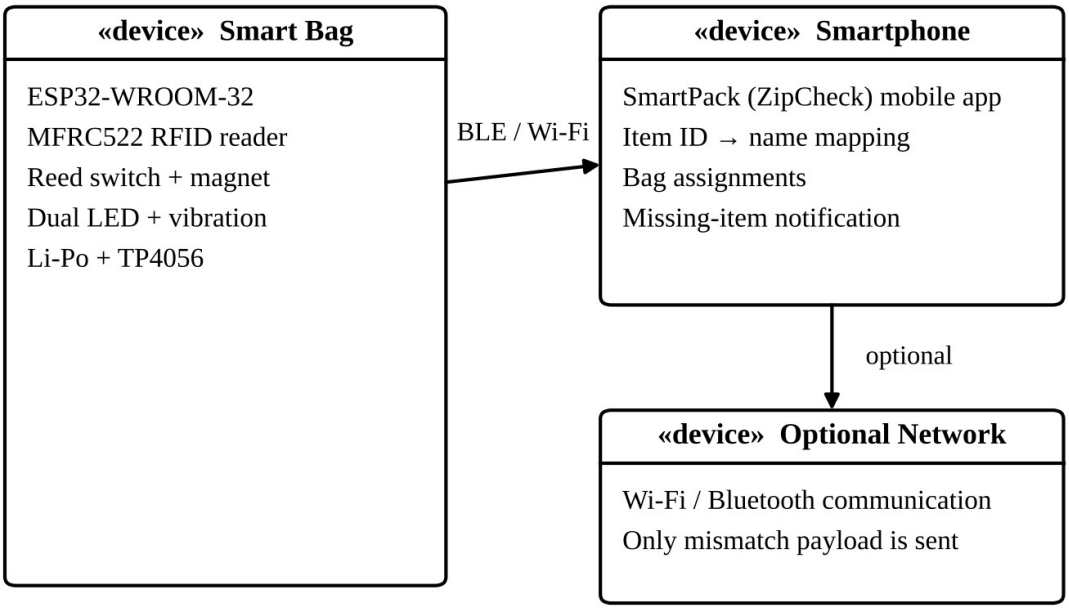


Figure 6: UML deployment diagram

9.7 State Machine Diagram

Figure 7 shows the states of the system and the events that move it from one state to the next: deep sleep, closure detected, scanning, comparing, then either all present or missing item, and finally notify and alert.

UML 7 – State Machine Diagram

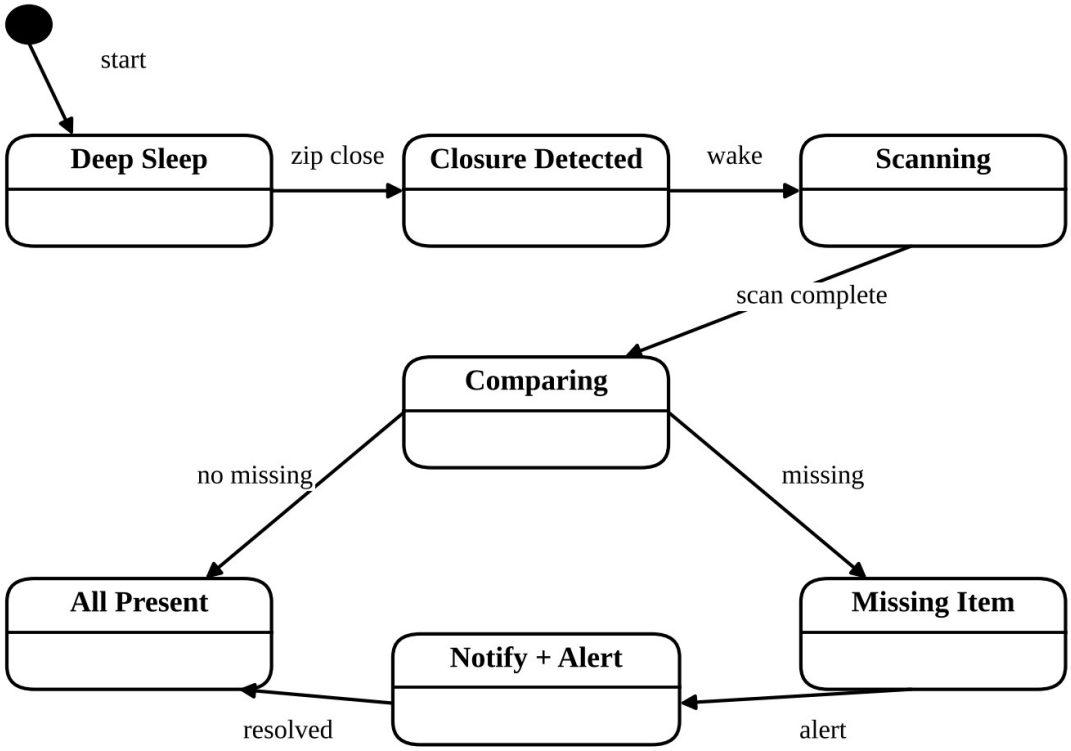


Figure 7: UML state machine diagram

9.8 Communication Diagram

Figure 8 shows the same story as the sequence diagram, but with numbered messages between the objects: the user closes the bag, the reed switch wakes the ESP32, the ESP32 asks the MFRC522 to scan, the tag IDs come back, the result goes to the alert module, the missing IDs go to the mobile app, and the app notifies the user.

UML 8 – Communication Diagram

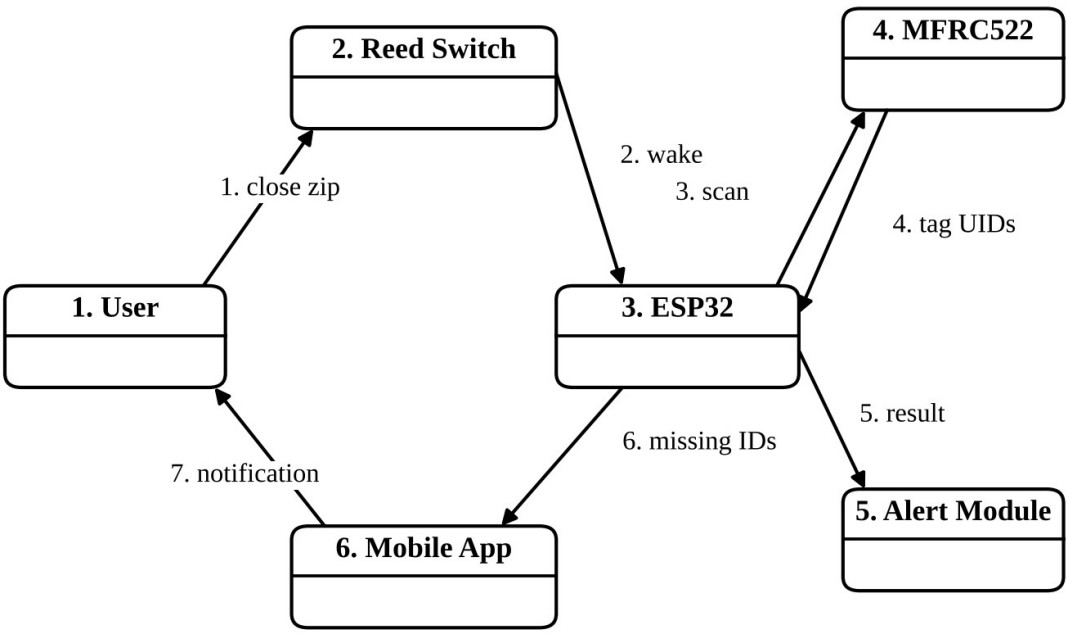


Figure 8: UML communication diagram

9.9 Differences Between the Design-Stage Diagrams and the Built Prototype

The diagrams above show the original closure-triggered design. While building the prototype the design was improved. Table 6 lists the main differences. The architecture and the algorithms of the built prototype are shown in Sections 10 and 11.

Table 6: Design-stage diagrams compared with the built prototype

Point	Design-stage diagrams	Built prototype
Readers	One RFID reader for the whole bag	One reader for each compartment (two in the prototype)
Closure sensing	One reed switch on the zip	One reed switch for each compartment
What is scanned	All tags scanned for 2–3 s after closure	Each scan is a crossing and toggles the item’s state; closure checks the states
Power mode	Deep sleep, wake on closure	Always on, so that scans and the phone link can be handled at any time
Audible alert	Vibration motor	Buzzer on GPIO 32
Phone link	BLE or Wi-Fi	BLE with a text command and event protocol
Phone features	Names, assignments, notification	Also tap-to-enrol, delete, rename, state correction and live status

10. ARCHITECTURE DESIGN

10.1 Three-Layer Architecture (Design Stage)

Figure 9 shows the architecture from the design stage. It has three layers. The **Bag Hardware Layer** senses, controls, gives local feedback and manages power. The **Control and Communication Layer** is the ESP32 firmware, which handles SPI, scanning, comparison and the wireless messages. The **Mobile Application Layer** stores the item library, the assignments and the names, and shows the notifications.

Architecture Design – Complete System Architecture

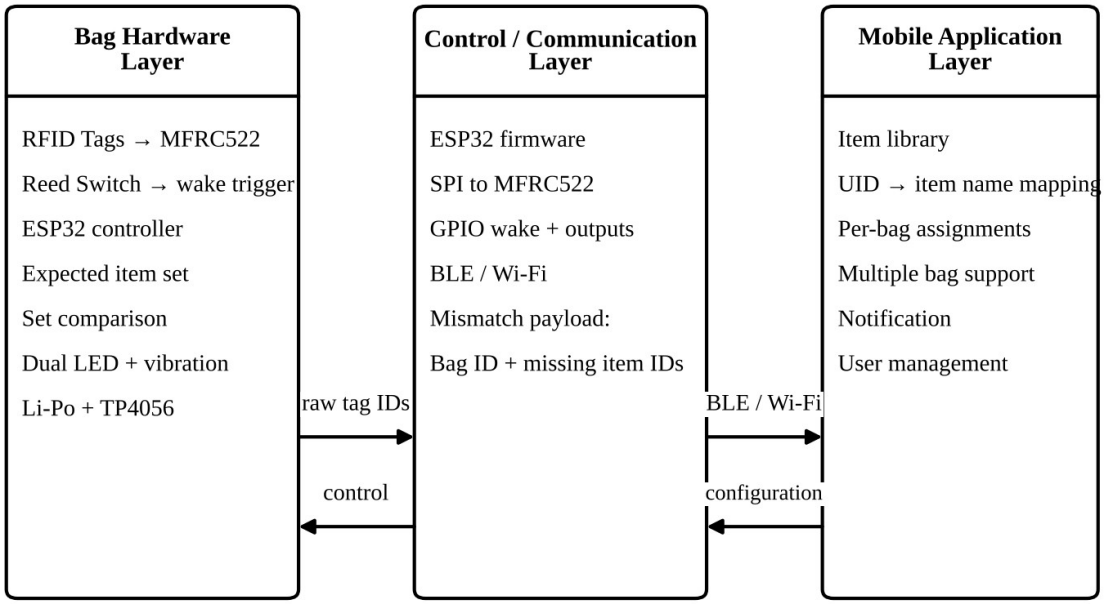


Figure 9: Complete system architecture (design stage)

10.2 Architecture of the Built Prototype

Figure 10 shows the architecture of the prototype. The hardware senses with two readers and two reed switches, the ESP32 firmware runs five functions (scan handling with the toggle, closure verification, flash storage, the indicator driver and the BLE server), and the local feedback is given by the two LEDs and the buzzer. The mobile application has four parts: the screens, the state store, the BLE service and local storage. The two sides talk over BLE using text commands and events. The battery chain feeds the ESP32.

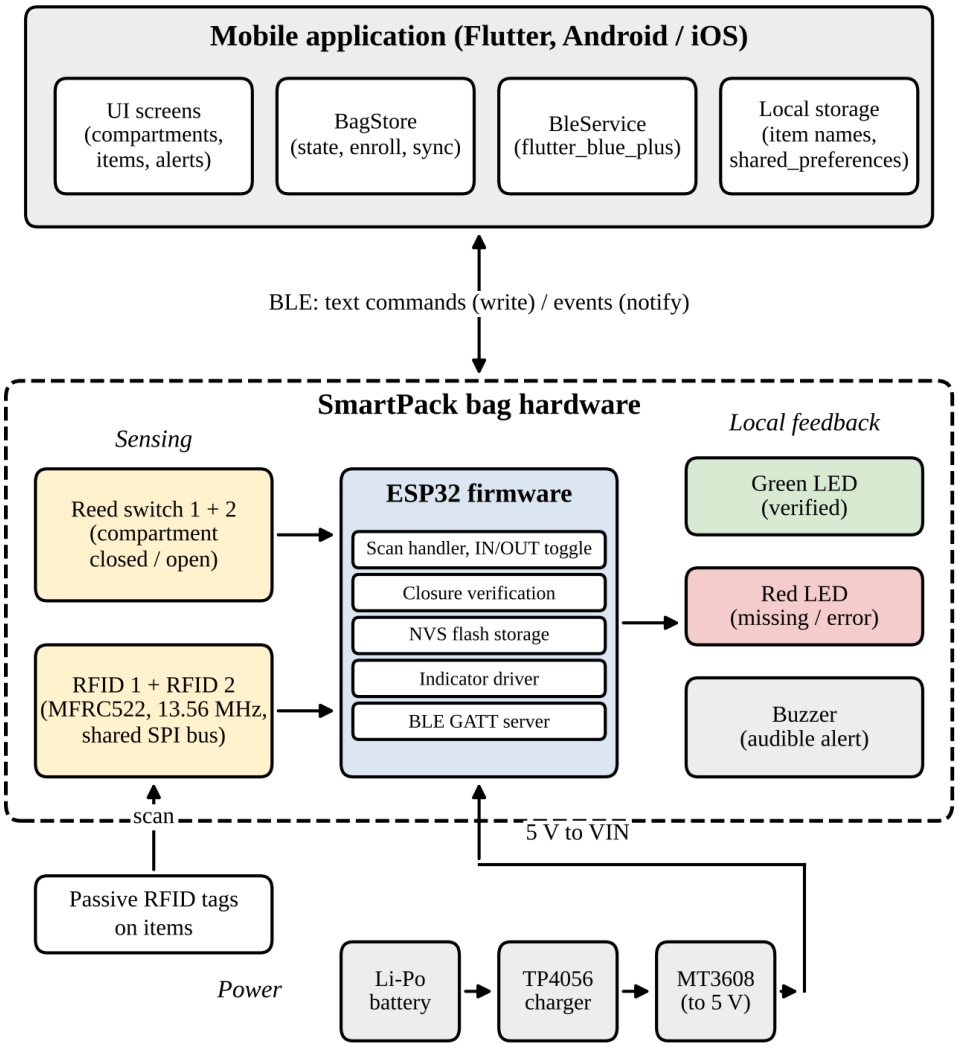


Figure 10: Architecture of the built SmartPack prototype

10.3 Main Components and Interfaces

Table 7: Main architecture components

Component	Responsibility	Interface
RFID tags	Identify each item by a unique ID	Passive RFID to reader
MFRC522 readers (two)	Read tag IDs at the compartment opening	SPI to ESP32 [15]
Reed switches (two)	Report whether a compartment is closed	GPIO input with pull-up
ESP32	Runs the firmware, the toggle, the verification and the BLE server [16]	SPI, GPIO, BLE
LEDs and buzzer	Give immediate local feedback	GPIO output
Mobile application	Keeps names, manages items and shows alerts	BLE [17]
Li-Po, TP4056, MT3608	Rechargeable power for the whole bag	Power rails

10.4 Data Architecture

Data is divided between the bag and the phone on purpose. The bag stores, for each compartment, a list of tag IDs and an inside flag for each of them, in the ESP32 flash memory. The phone stores the compartment names, the item names, the tag IDs and the flags in its own local storage. Names never go to the bag.

Table 8: Where each piece of data is stored

Data	Stored on	Reason
Tag IDs of assigned items	Bag and phone	The bag needs them to verify; the phone needs them to show names
Inside/outside state	Bag and phone	Bag decides; phone shows and can correct
Item names and compartment names	Phone only	Privacy: the bag reveals nothing about its owner’s items
Wiring table and limits	Firmware	Fixed at build time

11. ALGORITHMS & FLOWCHARTS

11.1 Design-Stage Algorithm and Flowchart

The design-stage algorithm is a set comparison. The expected set holds the IDs assigned to the bag, the detected set holds the IDs read during the scan after closure, and the missing set is the difference. The steps are:

- Load the expected set for the bag and wait in a low-power state.
- When the reed switch shows that the bag is closed, wake the ESP32 and power the reader.
- Scan for about two to three seconds and store the detected IDs.
- Calculate the missing set as the expected set minus the detected set.
- If the missing set is empty, show the green LED and a short buzz. Otherwise show the red alert and send the missing IDs to the app.
- Return to the low-power state.

Figure 11 shows this algorithm as a flowchart.

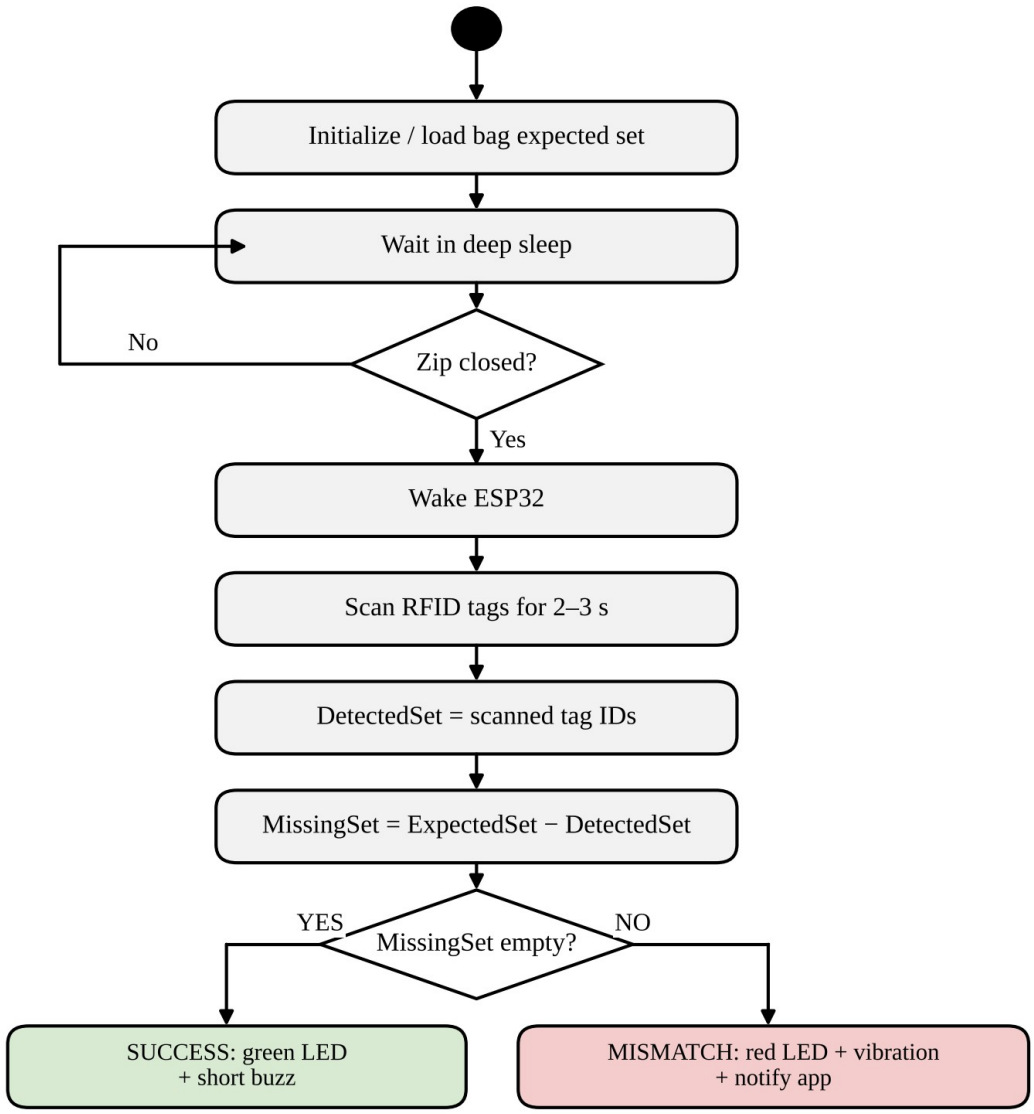


Figure 11: Flowchart of the design-stage verification algorithm

11.2 Algorithm 1: Toggle-Based Item Tracking

In the built prototype each tag read at a compartment's reader is one crossing of the opening. Figure 12 shows the two states of an item. A new item starts as outside. Each accepted scan flips the state. A tag that is not assigned to the compartment is reported as unknown and changes nothing.

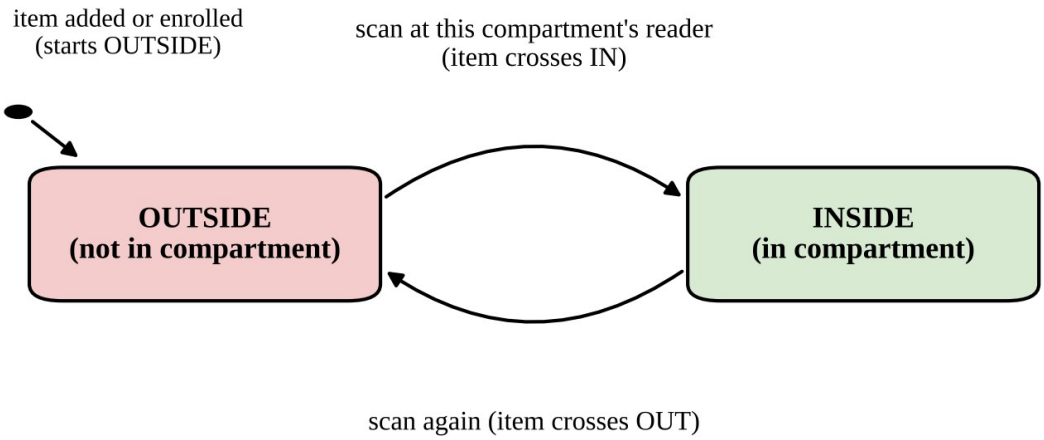


Figure 12: Inside/outside state of a tracked item

Four rules keep the toggle reliable. After a tag is read, a halt command is sent to it, so it must leave the field before it can be read again. The same tag on the same reader is ignored for 1.5 seconds. Scans at a closed compartment are ignored. And the app can force a state with a SET command if the user and the bag ever disagree.

11.3 Algorithm 2: Closure Verification

A reed switch level must stay steady for 60 milliseconds before it is accepted. When the accepted change is open to closed, the compartment is verified. Let A_c be the set of items assigned to compartment c and s_i the state of item i. The missing set is:

$$M_c = \{ i \in A_c : s_i = \text{OUTSIDE} \} \qquad (12.1)$$

If A_c is empty, there is nothing to check and no alert is given. If M_c is empty, the green LED lights for 1.5 seconds with one short beep. Otherwise the red LED lights for about 2.8 seconds with three beeps, one MISS message is sent for each missing item, and a final VERIFY message gives the count. Figure 13 shows the complete flow.

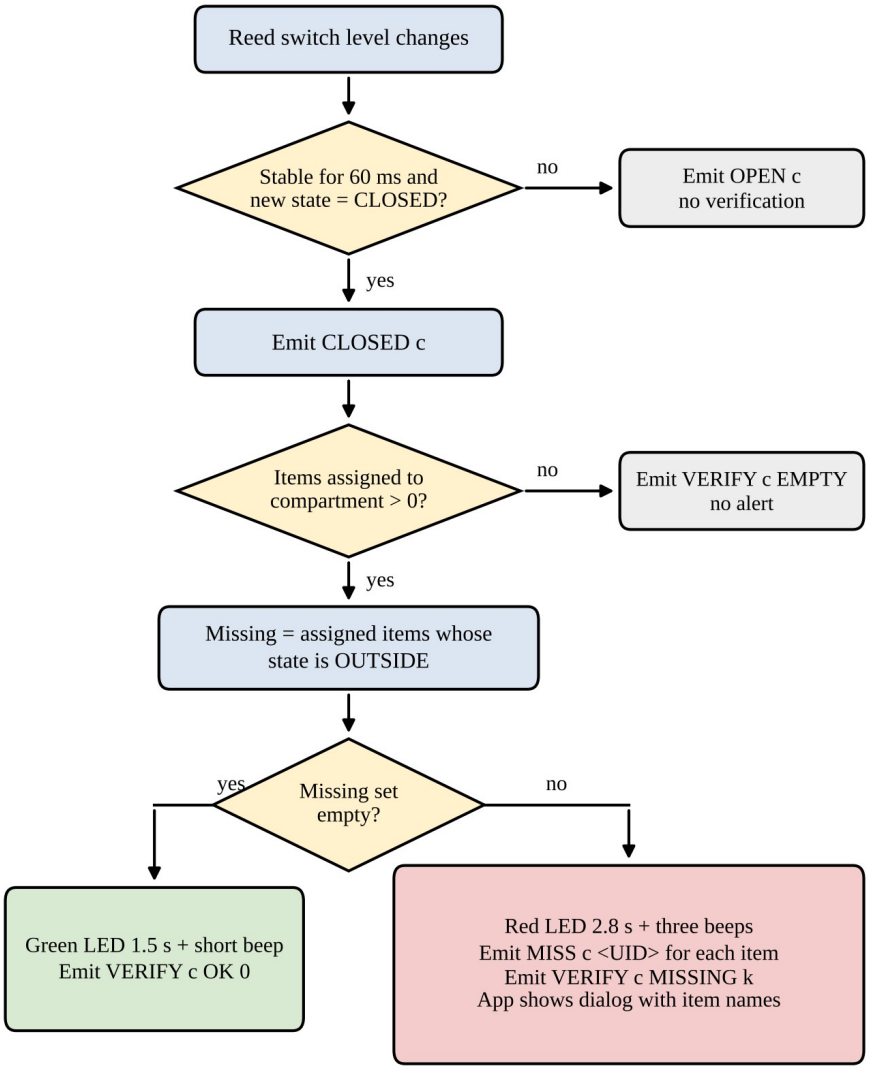


Figure 13: Flowchart of the closure verification of the built prototype

11.4 Algorithm 3: Tap-to-Enrol

To add an item, the user names it in the app. The app sends an ENROLL command, the user taps the tag on the reader, and the bag replies with the tag ID. Figure 14 shows the sequence. If the tag is already assigned to that compartment, the bag replies ENROLL_DUP. If no tag is scanned within 30 seconds, the bag replies ENROLL_TIMEOUT.

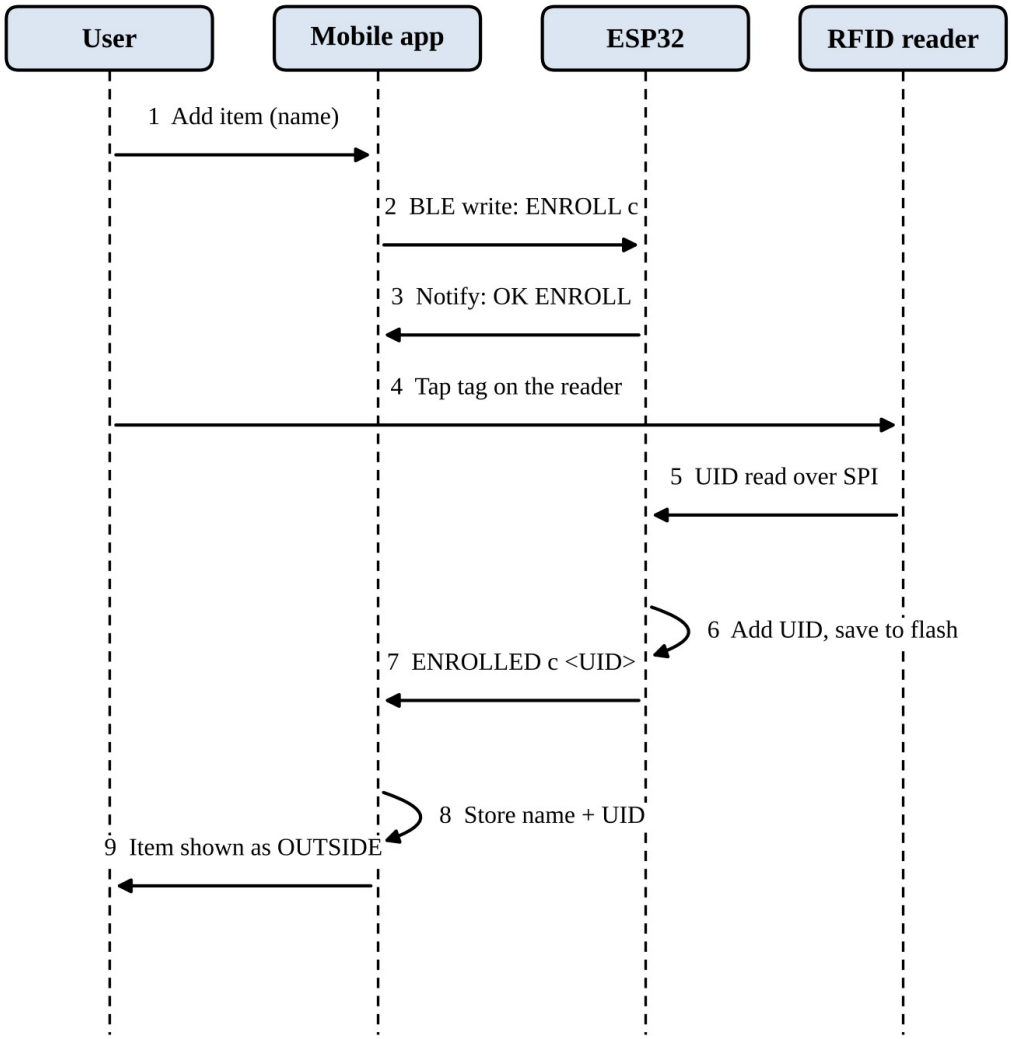


Figure 14: Sequence of adding an item by tap-to-enrol

11.5 Main Loop of the Firmware

The pseudocode below shows the main loop. Every part of it is short, so the loop runs many times each second.

```
setup():
    load saved configuration from flash
    reset both readers; read MFRC522 version (fault if 0x00 / 0xFF)
    adopt reed states silently; start BLE server
loop():
    handle commands from BLE and serial
    for each compartment c:
        debounce reed(c)
        on stable CLOSED edge: verify(c)
    every 60 ms, poll the next reader r:
        if a tag UID is read and it is not a repeat within 1.5 s:
            if enrolling: add UID to the requested compartment
            else if r is closed: report IGNORED_CLOSED
            else: toggle state, save, report IN / OUT / UNKNOWN
    enrol timeout (30 s); update LED/buzzer; send queued BLE events
```

11.6 Timing Settings and Cost

Table 9: Timing and size settings of the firmware

Setting	Value	Purpose
Reed debounce time	60 ms	Ignore contact bounce
Reader polling interval	60 ms per reader turn	Alternate between readers
Same-tag cooldown	1.5 s	One reading per presentation
Enrol timeout	30 s	Leave enrol mode if no tag is shown
Items per compartment	16	Limit of the stored list
Compartments	2 (build setting)	Prototype size

With at most sixteen items in a compartment, each scan and each verification needs only a short search through a small list, so the processing cost is very small compared with the time spent waiting between readings.

12. HARDWARE DEVELOPMENT

12.1 Hardware Overview and Components

The hardware has four jobs: it reads tags, senses the closing of each compartment, shows alerts, and supplies power. Table 10 lists the parts and their roles.

Table 10: Hardware components

Component	Qty	Role
ESP32-WROOM-32 development board	1	Controller with SPI, GPIO and built-in BLE [16]
MFRC522 RFID reader (13.56 MHz)	2	One for each compartment; reads tag IDs over SPI [15]
Passive 13.56 MHz RFID tags (stickers)	10	One for each tracked item
Reed switch with magnet	2	One for each compartment; closed contact means the compartment is closed
Green LED and red LED	1 + 1	Verified / missing or error
Resistors 220–330 Ω	2	One in series with each LED
Active buzzer (small, low current)	1	Audible alerts
Li-Po battery	1	Portable power
TP4056 charging module	1	Charges and protects the battery
MT3608 boost converter	1	Raises the battery voltage to about 5 V
Perfboard, wires, enclosure	–	Assembly inside the bag

12.2 Wiring Diagram of the Two-Compartment Prototype

Figure 15 shows how the parts are connected. The two MFRC522 readers share the three SPI lines (SCK, MISO and MOSI), and each has its own slave-select (SS) and reset (RST) line. Each reed switch connects one GPIO to ground. Each LED is connected through a resistor, and the buzzer connects to GPIO 32. Both readers are powered from the 3V3 pin. The power chain at the bottom feeds the VIN pin with about 5 V.

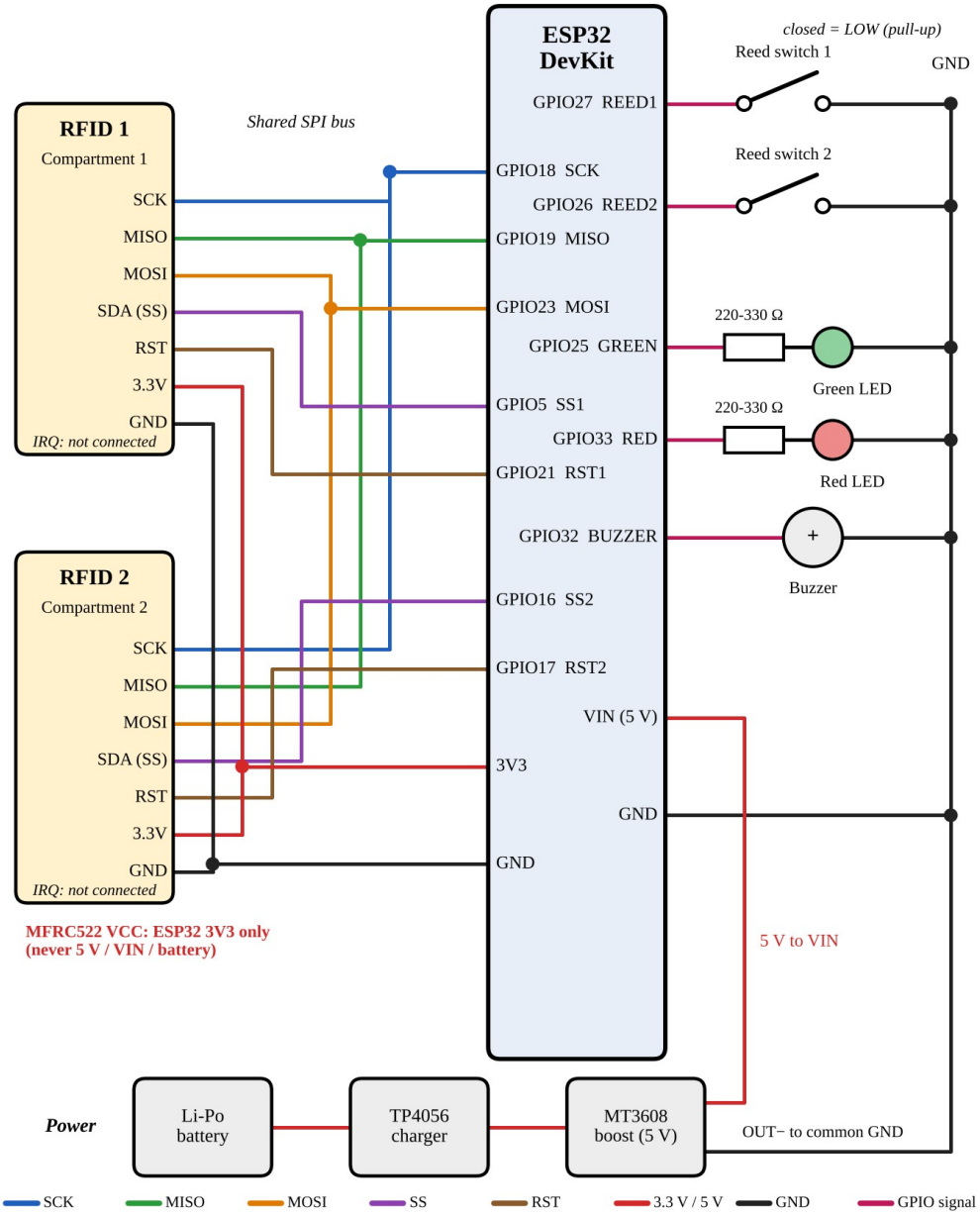


Figure 15: Wiring diagram of the two-compartment SmartPack prototype

12.3 Pin Assignment

Table 11: ESP32 pin assignment

Component	Signal	GPIO	Note
RFID 1 (compartment 1)	SDA / SS / CS	5	Reader 1 only
RFID 1 (compartment 1)	RST	21	Reader 1 only
RFID 2 (compartment 2)	SDA / SS / CS	16	Reader 2 only
RFID 2 (compartment 2)	RST	17	Reader 2 only
Shared SPI bus	SCK	18	Both readers
Shared SPI bus	MISO	19	Both readers

Component	Signal	GPIO	Note
Shared SPI bus	MOSI	23	Both readers
Reed switch 1	Signal	27	Other wire to GND; closed = LOW
Reed switch 2	Signal	26	Other wire to GND; closed = LOW
Green LED	Anode (+)	25	Through 220–330 Ω
Red LED	Anode (+)	33	Through 220–330 Ω
Buzzer	Positive (+)	32	Small low-current active buzzer
MFRC522 (both)	VCC	3V3	Never 5 V, VIN or the battery

The IRQ pin of both readers is not connected. GPIO 16 and GPIO 17 are used for the second reader, so it should be checked that the exact board in use does not use these pins for another purpose. This pin map is for a classic ESP32-WROOM development board.

12.4 Power Supply

The Li-Po battery is connected to the TP4056 module, which charges it and protects it. The output of the TP4056 goes to the MT3608 boost converter. The MT3608 must be adjusted with a multimeter to about 5.0 V **before** it is connected to the ESP32. Its output feeds the VIN pin of the ESP32, and the ESP32's 3.3 V pin then supplies both RFID readers.

12.5 Safety Rules

- Connect the VCC of the MFRC522 readers only to the ESP32 3V3 pin. Never connect it to 5 V, VIN, the MT3608 output or the battery.
- Set the MT3608 to about 5 V with a multimeter before connecting it to the ESP32.
- Use a 220 Ω or 330 Ω resistor with each LED.
- Join all grounds together.
- Both readers share SCK, MISO and MOSI, but each must have its own SS pin and its own RST pin.
- A small low-current buzzer can be driven directly from the GPIO. A larger buzzer needs a transistor driver.

12.6 Building the Working Model

The prototype was built in the following order: the ESP32 and one reader were wired first and tested; the second reader and the reed switches were added; the LEDs and the buzzer were added; and then the power chain. Each step was tested before the next one. The

circuit is built on breadboards that are fixed to a cardboard base on the front pocket of a backpack, so that the wiring can be changed easily during testing. Figure 16 shows the finished working model.

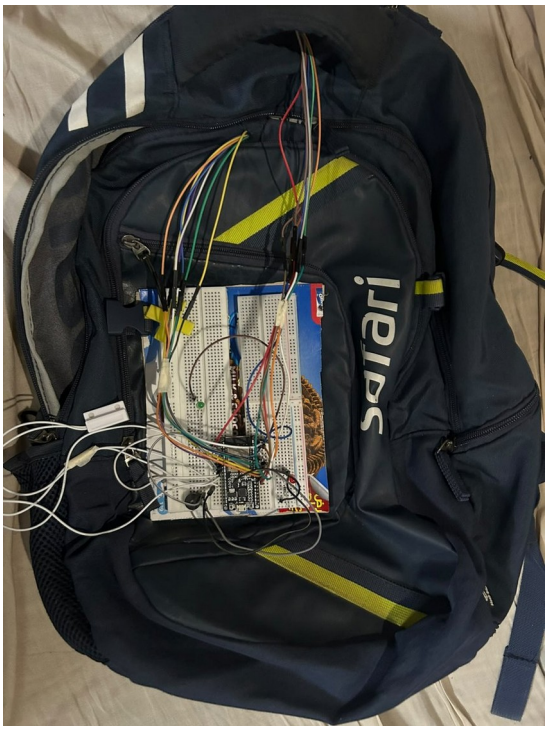


Figure 16: The assembled SmartPack working model (front view)

Figure 17 shows a close view of the electronics. The ESP32 board, the LEDs, the buzzer and the boost converter are joined with jumper wires. The red LED is lit in this photograph.

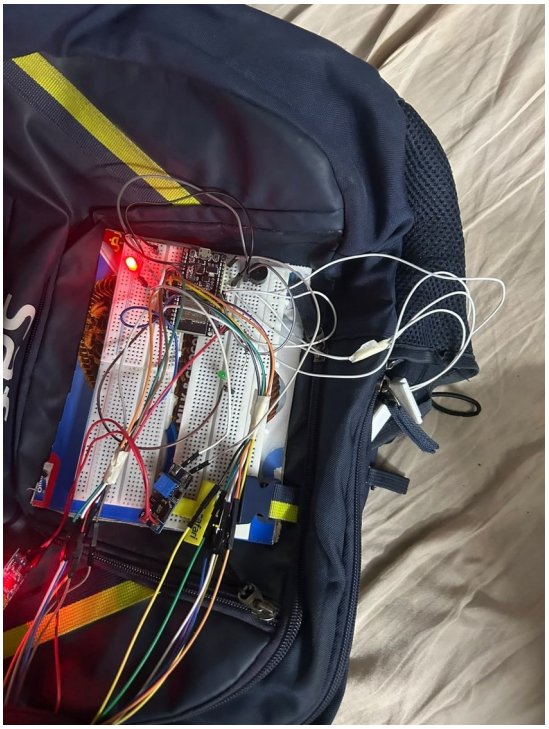


Figure 17: Electronics of the working model on breadboards

Each compartment has its own RFID reader. Figure 18 shows the reader of a compartment fixed inside it, with its wires coming out through the zip. Items that carry a tag are brought near the reader as they go into or out of the compartment.



Figure 18: RFID reader (RFID-RC522) fixed inside a compartment

Figure 19 shows the reed switch and its magnet on the zip of a compartment. The reed switch is fixed on one side of the zip and the magnet on the other side, so the two come together when the zip is closed. This is how the bag knows that the compartment is closed.



Figure 19: Reed switch and magnet fitted on the zip of a compartment

12.7 Adding More Compartments

The prototype has two compartments, but the design allows more. Each added compartment needs one more reader, one more reed switch and three free GPIO pins. The steps are:

- Connect another MFRC522 reader to the shared SPI bus with its own SS and RST pins, and another reed switch to a free GPIO.
- Add one row {SS, RST, reed} for the new compartment to the wiring table in the firmware.
- Raise the compartment-count setting in the firmware and the matching constant in the app. The firmware will not build if the table and the count disagree.

There are limits. Each compartment uses three GPIO pins, so the board’s free pins limit the number. The input-only pins GPIO 34 to 39 have no internal pull-up and would need an external resistor for a reed switch. The readers are polled in turn, so each reader is polled less often when there are more of them. Neighbouring readers may also read each other’s tags if the compartments are very close, and this needs a test. Configurations with more than two compartments have not yet been built.

13. SOFTWARE DEVELOPMENT

13.1 Development Tools

Table 12: Development environment

Item	Version or setting
Firmware framework	Arduino framework for ESP32 (Arduino IDE; the same files also build with PlatformIO), board “ESP32 Dev Module”
RFID library	MFRC522v2 (Arduino_MFRC522v2) [18]
BLE and storage	ESP32 BLE library and the Preferences library for flash storage
Mobile application	Flutter 3.38.9 (stable) with Dart 3.10.8 [19]
App packages	flutter_blue_plus ^1.35.0 [20], provider ^6.1.2, shared_preferences ^2.2.3
Android build	Android SDK 36.1.0, NDK 28.2.13676358, JDK 21 (from Android Studio)
Source control	Git repository with the firmware, the app, the protocol document and the pin map

13.2 Firmware

13.2.1 Structure

The firmware is divided into small parts, each with one job:

- **Core logic:** the tag ID type, the item list and state of each compartment, the toggle, the closure check and the command parser. It does not depend on the Arduino hardware, so it can be tested on a PC.
- **Main loop:** reed debouncing, reader polling, enrol mode and command handling, driven by a wiring table with one row for each compartment.
- **Indicator driver:** plays the LED and buzzer patterns without blocking the loop.
- **Storage:** saves the whole configuration and all item states to the ESP32 flash after every change, so the bag keeps its items after a power cycle.
- **BLE link:** a GATT service with one write characteristic for commands and one notify characteristic for events.
- **Serial console:** accepts the same text commands over USB for testing without the app.

13.2.2 Local Feedback Patterns

Table 13: LED and buzzer patterns

Event	LED	Buzzer
Item crossed in	Green, one short flash	One short beep
Item crossed out	Green, two short flashes	Two short beeps
Compartment verified	Green, 1.5 s	One 150 ms beep
Items missing at closure	Red, about 2.8 s	Three 200 ms beeps
Unknown tag	Red, one short flash	One short beep
Tag enrolled	Green, three short flashes	Three short beeps
Reader not responding at start-up	Red, two long flashes	Two long beeps

13.3 BLE Communication Protocol

The bag advertises as “SmartPack” with a custom service and requests a 185-byte MTU. Each message is one short line of text. Text was chosen because it is easy to read in a serial monitor and easy to build and parse in the app. Commands are written to one characteristic and events arrive as notifications on another [17]. Compartments are numbered from 1, and tag IDs are written in upper-case hexadecimal.

Table 14: Commands from the app to the bag

Command	Effect
ENROLL c	Add the next scanned tag to compartment c (30 s timeout)
CANCEL	Leave enrol mode
ADD c UID	Assign a tag to compartment c
DEL c UID	Remove a tag from compartment c
CLEAR c	Remove all items of compartment c
SET c UID 0|1	Force an item’s state (0 outside, 1 inside)
LIST c	Reply with one ITEM line for each item of compartment c
STATUS	Reply with one STATE line for each compartment

Table 15: Events from the bag to the app

Event	Meaning
READY	The bag has started
IN c UID, OUT c UID	An item crossed into or out of compartment c
UNKNOWN c UID, IGNORED_CLOSED c UID	Tag not assigned, or scanned while the compartment was closed
OPEN c, CLOSED c	Debounced reed switch change

Event	Meaning
MISS c UID … VERIFY c MISSING k	At closure: each missing item, then the count
VERIFY c OK 0, VERIFY c EMPTY 0	At closure: all present, or nothing assigned
ENROLLED, ENROLL_DUP, ENROLL_TIMEOUT	Results of an enrol request
ITEM, STATE	Replies to LIST and STATUS
OK cmd, ERR reason	Command accepted or rejected

13.4 Mobile Application

13.4.1 Architecture

The application is written in Flutter, so one code base builds for Android and iOS. It has four layers:

- **Protocol layer:** a parser that turns each event line from the bag into a typed event.
- **BLE service:** scans for the SmartPack service, connects, requests a larger MTU, subscribes to events and writes commands.
- **State layer:** holds the compartments and items, handles every event, runs the enrol flow, synchronises with the bag and raises the missing-item alert.
- **Screens and storage:** one card for each compartment, and local storage for names, tag IDs and states.

13.4.2 What the User Can Do

- **Connect:** tap the Bluetooth icon, choose the SmartPack from the scan list and connect.
- **Add an item:** open a compartment, tap Add item, type a name and tap the tag on the reader.
- **See status:** every item shows whether it is inside or outside, and each compartment shows whether it is open or closed.
- **Correct a state:** use the switch on an item’s row if the app and the bag disagree.
- **Delete an item:** tap the bin icon on the item and confirm. The bag and the phone must be connected, so that both remove the item.
- **Rename a compartment:** tap its title.
- **See alerts:** when a compartment is closed with items missing, a dialog names each missing item and the phone vibrates.

13.4.3 Synchronisation

Each time the app connects, it asks the bag for the item list of every compartment. Items that the phone knows but the bag has lost are sent again. The inside/outside state is taken from the bag. A tag that the bag knows but the phone has no name for appears as “Unnamed tag”, so that it can be renamed.

Figure 20 shows the application on an Android phone while it is connected to the bag. Each compartment is shown as a card with its items, their tag IDs and their inside/outside state.

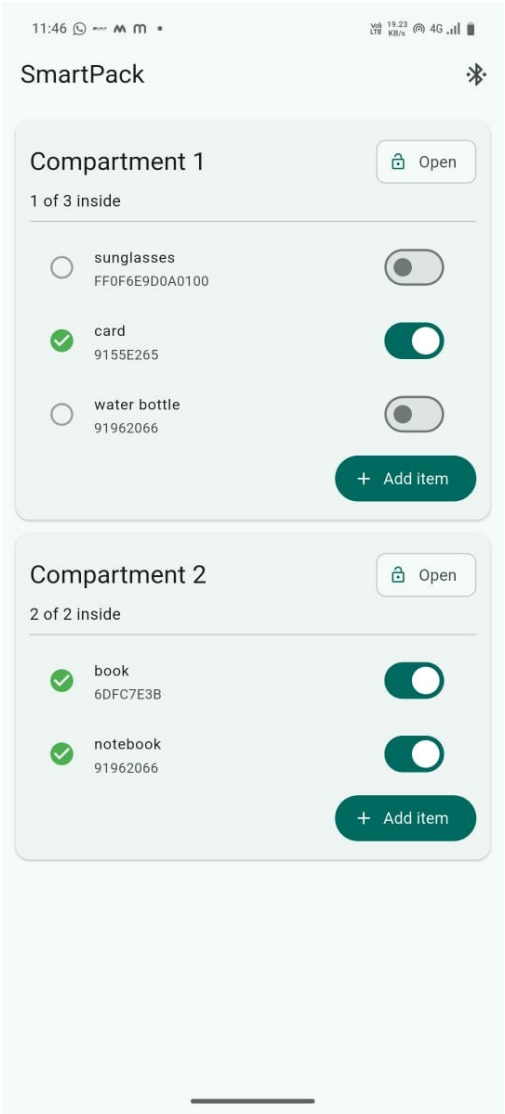


Figure 20: Screen of the SmartPack mobile application

13.5 Software Testing Done So Far

Table 16: Software tests

Test	What it covers	Result
Core logic tests (PC)	ID conversion, duplicate assignment, independent compartments, toggle, missing set, capacity, state forcing, command parser including bad commands	All passed (run for 2, 3 and 4 compartments)
Protocol parser tests (Dart)	Tag, reed, verify and reply events, and malformed lines	All four passed
Firmware build	Compiled for ESP32 Dev Module in the Arduino IDE	Succeeded; 86% of program storage and 13% of dynamic memory in the first build (to be measured again for the final code)
Wiring-table check	Build stops if the table has the wrong number of rows	Checked with 2 and 3 compartments

14. CLOUD / DATA ANALYTICS / COMPUTER SCIENCE INTEGRATION

14.1 Cloud Integration

The prototype does **not** use a cloud service, and this is a deliberate choice. The check must work when the user is in a hurry and may have no internet. Keeping everything on the bag and the phone also protects privacy, because no item data leaves the two devices. The only link is a short-range Bluetooth connection.

A cloud service could be added later for tasks that do not need to be instant, such as keeping a backup of the item names when the user changes phones, or sharing a bag between family members. Such a service would send names and states only after the user agrees.

14.2 Computer Science Concepts Used

Table 17: Computer science concepts used in the project

Concept	Where it is used
Embedded systems and real-time polling	The ESP32 firmware polls readers and switches in a loop with fixed timing
Finite state machines	Each item is in one of two states; the system moves between states on events (Section 11)
Set theory	The missing set is the set of assigned items that are not inside
Data structures	Fixed-size lists of items for each compartment, with a state flag
Communication protocols	SPI between the ESP32 and the readers; BLE with a text command and event protocol
Concurrency	BLE callbacks run in another task, so commands go through a queue to the main loop
Persistent storage	Flash storage on the bag; local storage on the phone
Mobile software architecture	Layers for protocol, BLE, state and screens in the Flutter app
Software testing	Unit tests of the core logic and the parser; planned bench trials

14.3 Data Analytics Possibilities

The bag already produces simple events: when each compartment is opened and closed, and which items were scanned in and out. In future, the phone could keep a private history of these events and show useful patterns, for example which item is most often missing at closure, or at what time of day the user usually forgets something. This is not part of the prototype. If it is added, the data should stay on the phone unless the user chooses otherwise.

14.4 Privacy and Security

Privacy is helped by keeping names on the phone. The Bluetooth link of the prototype does not yet use pairing or encryption, so a nearby device could connect and send commands. Before real-world use, the link should use bonding with encryption, and a physical button should be needed to allow a new item to be enrolled.

15. FUNCTIONAL DEMONSTRATION

15.1 Demonstration Setup

For the demonstration the following are needed: the working SmartPack bag with at least two tagged items in each compartment, a phone with the SmartPack application installed, and a computer with a serial monitor (optional, to show the raw messages at 115200 baud). The bag is switched on and the app is connected to it.

15.2 Demonstration Script

Table 18 gives the steps to show, with the response that the system is designed to give at each step. The photographs and the application screen in this report show the working model during these steps.

Table 18: Demonstration steps and expected responses

Step	Action	Expected response
1	Switch on the bag	READY message; no alarm
2	Connect the app to the bag	App shows both compartments with their items
3	Add an item to compartment 1: type a name, tap its tag on reader 1	Three green flashes; item appears in the app as outside
4	Scan the item once while compartment 1 is open	One green flash and beep; app shows the item as inside
5	Close compartment 1 with all items inside	Green LED for 1.5 s and one beep; app confirms
6	Open the compartment, scan one item again (it goes out), close it	Red LED about 2.8 s with three beeps; app dialog names the missing item
7	Scan the item again (back in), close the compartment	Green LED; no missing items
8	Scan a tag that is not assigned to the compartment	One short red flash and beep; nothing changes
9	Repeat steps 4 to 7 on compartment 2	Same behaviour, independent of compartment 1
10	Delete an item in the app (connected)	Item disappears from the app; the bag no longer tracks it
11	Switch the bag off and on	Items and their states are restored

15.3 Observations

The working model was demonstrated with the phone connected to the bag over Bluetooth. Figure 20 shows the application at that time. The **Add item** buttons are active in the figure, and the application keeps them switched off when it is not connected, so this shows that the phone and the bag were connected.

The screen shows that all items were added to the two compartments by tapping their tags. Compartment 1 holds three items: sunglasses, a card and a water bottle. It shows “1 of 3 inside” because only the card has been scanned in, while the sunglasses and the water bottle are still outside. Compartment 2 holds a book and a notebook, and both are inside, so it shows “2 of 2 inside”. Both compartments are shown as Open, so no closure check was running at that moment.

The tag ID of every item is shown below its name. These IDs were read by the RFID readers on the bag and sent to the phone, so the readers, the Bluetooth link and the tap-to-enrol steps all worked. The tag of the sunglasses has a longer 7-byte ID, and the other tags have 4-byte IDs, so the system accepts both sizes. The water bottle in compartment 1 and the notebook in compartment 2 use the same tag ID, yet one is outside and the other is inside. This shows that each compartment keeps its own list and its own states.

The photographs show the hardware that produced these results. The RFID reader is fixed inside a compartment (Figure 18), the reed switch and its magnet are on the zip (Figure 19), and the ESP32 board with the LEDs and the buzzer is on the breadboards (Figure 17), where the red LED is lit.

Two points need improvement. The circuit is on breadboards with jumper wires, and many wires hang outside the bag, so a wire can come loose when the bag is moved. In the next version the circuit should be soldered on a perfboard, fixed inside the bag and covered. The reader is also placed loosely inside the compartment, so the distance between a tag and the reader can change from one scan to the next. It should be fixed at the opening of the compartment, and the readers of neighbouring compartments should be tested for cross-reading.

16. VALIDATION & DEPLOYMENT

16.1 Validation Completed So Far

Table 19: Validation status

Item	Method	Outcome
Core logic	PC unit tests (toggle, missing set, parser), run for 2, 3 and 4 compartments	All passed
Application parser	Four Dart unit tests	All passed
Firmware build	Compiled for ESP32 Dev Module in Arduino IDE	Succeeded (86% of program storage, first build)
Compartment-count check	Firmware sources type-checked for 2 and 3 compartments; build stops on a wrong table	Passed; not tried on hardware with more than 2
Firmware on the hardware	Upload to the ESP32 and test with the working model	Running on the working model; the readers returned the tag IDs that appear in the app (Figure 20)
Application build and install	Flutter build and installation on an Android phone	Built, installed and running (Figure 20)
End-to-end test	BLE connection, tap-to-enrol and status display with the working model	The app connected to the bag, enrolled five items in two compartments and showed their inside/outside states (Figure 20)

16.2 Planned Further Measurements

The demonstration shows that the system works. To measure how well it works, further trials with repeated scans and closures are planned. The following will be measured:

- **Toggle accuracy:** how often a scan gives the expected in or out change, and how often a scan is missed or counted twice.
- **Closure correctness:** how often the alert at closure matches what is really inside, including false alarms and missed detections.
- **Delay:** the time from closing a compartment to the LED turning on.
- **Reading distance:** the largest distance at which a tag is read reliably, and whether neighbouring readers read each other’s tags.
- **Battery run time:** the hours of operation on one charge, because the ESP32 stays awake.

16.3 Known Limitations and Risks

- **The check uses scan records.** The closure check compares the recorded state of each item. If an item is placed without being scanned, it is reported missing. If an item is scanned in but not placed, the check cannot tell. A confirmation scan of the inside of the compartment at closure would remove this problem.
- **Toggle drift.** One missed or extra scan inverts an item's state until it is corrected. The cooldown, the halt command and the correction switch in the app reduce this.
- **Always-on power use.** The ESP32 stays awake, so the battery lasts shorter than in the original sleeping design. The planned power test will measure this.
- **Short reading range and metal.** HF tags must be brought close to the reader, and metal or foil in the bag may weaken the reading.
- **Open Bluetooth link.** The link has no pairing or encryption yet.
- **Alerts need the app open.** Missing-item dialogs appear while the app is running and connected.

16.4 Deployment

The steps to put SmartPack into use are:

- **Prepare the hardware.** Assemble the circuit as in the wiring diagram, set the MT3608 to about 5 V before connecting the ESP32, and fix the readers and reed switches at the compartment openings with the magnets on the matching flaps.
- **Load the firmware.** Open the Arduino sketch, select the ESP32 Dev Module and the correct port, upload, and open the serial monitor at 115200 baud to check that both readers report a version.
- **Install the app.** Build the release APK with Flutter, copy it to the phone and install it (allow installation from unknown sources). Allow the Bluetooth permissions when asked.
- **Connect and add items.** Connect the app to the bag, then add each item with tap-to-enrol and scan it once to put it inside.
- **Use daily.** Scan an item as it goes in or out, and close the compartment. Charge the battery through the TP4056 when it is low.

16.5 Conclusion

SmartPack checks the contents of a bag at the moment each compartment is closed. Each compartment has its own reader and reed switch, each scan flips an item between inside and outside, and a missing item gives an immediate local alert and a message to the phone. The mobile application adds, lists, deletes and corrects items, and keeps all names

on the phone. The decision logic and the application parser are covered by automated tests, and the design is table-driven so that more compartments can be added. The further measurements described in Section 16.2 are the remaining step to find the accuracy, delay and battery life of the working model.

16.6 Future Work

- Add a confirmation scan inside the compartment at closure.
- Reduce power use with duty-cycled polling and light sleep.
- Add background notifications and automatic reconnection in the app.
- Add Bluetooth pairing with encryption and a physical enrol button.
- Build and test more than two compartments, and let the app discover the number of compartments from the bag.
- Support several bags in one application and the same item in different bags.
- Complete a wider prior-art search for the novelty claim.

REFERENCES

[1] G. Borriello, W. Brunette, M. Hall, C. Hartung, and C. Tangney, "Reminding about tagged objects using passive RFIDs," in *UbiComp 2004: Ubiquitous Computing*, N. Davies, E. D. Mynatt, and I. Siio, Eds., Lecture Notes in Computer Science, vol. 3205. Berlin, Germany: Springer, 2004, pp. 36–53, doi: 10.1007/978-3-540-30119-6_3.

[2] H.-H. Hsu, C.-N. Lee, and Y.-F. Chen, "An RFID-based reminder system for smart home," in *Proc. 25th IEEE Int. Conf. Adv. Inf. Netw. Appl. (AINA)*, Singapore, 2011, pp. 264–269.

[3] A. K. Singh, S. Meshram, T. Gujar, and P. R. Wankhede, "Baggage tracing and handling system using RFID and IoT for airports," in *Proc. Int. Conf. Comput., Anal. Secur. Trends (CAST)*, 2016, pp. 466–470. [Online]. Available: https://ieeexplore.ieee.org/document/7915014/

[4] K. Guo, X. Xie, H. Qi, and K. Li, "A high time-efficient missing tag detection scheme for integrated RFID systems," *Sensors*, vol. 22, no. 12, Art. no. 4601, 2022, doi: 10.3390/s22124601.

[5] J. Hallberg, C. Nugent, R. Davies, and M. Donnelly, "Localisation of forgotten items using RFID technology," in *Proc. 9th Int. Conf. Inf. Technol. Appl. Biomed. (ITAB)*, 2009. [Online]. Available: https://ieeexplore.ieee.org/document/5394459/

[6] A. S. Chauhan and D. Nigam, "Smart bag based on RFID and Internet of Things," in *Proc. ICTCon 2021*, AIJR Proceedings, 2021, pp. 102–105. [Online]. Available: https://books.aijr.org/index.php/press/catalog/book/115/chapter/1127

[7] P. Kasun and S. M. Vidanagamachchi, "RFID based smart personal baggage system," in *Proc. Int. Res. Conf., Uva Wellassa Univ.*, Badulla, Sri Lanka, Feb. 2018. [Online]. Available: https://erepo.lib.uwu.ac.lk/items/1630e69d-9555-4f7b-9f99-d4db10028457

[8] R. Varma, P. Pavshe, A. Bhadane, and S. Pagare, "Multifunctional bag monitoring system," *Int. Res. J. Eng. Technol. (IRJET)*, vol. 5, no. 12, Dec. 2018. [Online]. Available: https://www.irjet.net/archives/V5/i12/IRJET-V5I12130.pdf

[9] S. M. Antad, O. Khanvilkar, C. Kharare, J. Kharat, and K. Kharat, "An IoT-based smart bag with integrated security system and rainwear," in *Proc. IEEE Int. Conf. Blockchain Distrib. Syst. Secur. (ICBDS)*, New Raipur, India, 2023, pp. 1–5, doi: 10.1109/ICBDS58040.2023.10346331.

[10] M. S. Mohd Yazed and F. Mahmud, "RFID-based interactive log bag for poor memory prevention," *Int. J. Integr. Eng.*, vol. 8, no. 1, 2016. [Online]. Available: https://journal.uthm.edu.my/index.php/ijie/article/view/1403

[11] M. K. Watfa, M. Kaur, and R. F. Daruwala, "An intelligent RFID system," *Int. J. Intell. Syst. Technol. Appl.*, vol. 10, no. 4, pp. 377–395, 2011.

[12] A.-M. V. M. Calo, T. H. O. Bato, C. J. Abragan, and E. S. Tutor, "Real-time detection and notification system of forgotten items using ultra high frequency RFID technology," *Indian J. Sci. Technol.*, vol. 11, no. 48, Dec. 2018.

[13] C. Gomez, J. Oller, and J. Paradells, "Overview and evaluation of Bluetooth Low Energy: An emerging low-power wireless technology," *Sensors*, vol. 12, no. 9, pp. 11734–11753, 2012, doi: 10.3390/s120911734.

[14] Object Management Group, "Unified Modeling Language (UML) specification." [Online]. Available: https://www.omg.org/spec/UML/

[15] NXP Semiconductors, "MFRC522 standard performance MIFARE and NTAG frontend," Product data sheet. [Online]. Available: https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf

[16] Espressif Systems, "ESP32 series datasheet." [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

[17] Bluetooth Special Interest Group, "Bluetooth core specification." [Online]. Available: https://www.bluetooth.com/specifications/specs/

[18] OSSLibraries, *Arduino_MFRC522v2* (software library), GitHub. [Online]. Available: https://github.com/OSSLibraries/Arduino_MFRC522v2

[19] Google, "Flutter documentation." [Online]. Available: https://docs.flutter.dev

[20] *flutter_blue_plus* (software package), pub.dev. [Online]. Available: https://pub.dev/packages/flutter_blue_plus